

Deep Learning Algorithms 1

An interview-focused guide to foundational neural networks, generative models, sequence architectures, language models, and computer-vision networks. Every chapter moves from intuition and mathematics to implementation, evaluation, interview questions, and project practice.

Notation: B = batch size, L/T = sequence length, C = channels, $H \times W$ = spatial size, d = hidden/embedding dimension, and V = vocabulary size.

Perceptron

1. Overview

The perceptron is an early binary classifier that uses a linear decision boundary and a step function. It is useful historically and conceptually because it explains linear separability, weight updates, and why multilayer networks became necessary.

2. Intuition

It is like a yes/no rule: if weighted evidence crosses a threshold, predict class 1 ; otherwise predict 0 or -1 .

3. Prerequisites

Vectors, dot product, binary classification, linearly separable data, learning rate, basic Python.

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Linear boundary	Hyperplane separates classes	Defines model capacity	$w_1 \cdot x_1 + w_2 \cdot x_2 + b = 0$	Linear separability
Step activation	Hard class decision	Non-probabilistic output	$1 \text{ if } z \geq 0 \text{ else } 0$	Not differentiable
Update rule	Correct mistakes only	Simple online learning	Move boundary toward correct class	Convergence theorem
Separability	Perfect linear split exists	Required for convergence	AND is separable, XOR is not	Classic question

5. Algorithm / Working Process

Initialize weights and bias. For each training example, compute prediction. If prediction is wrong, update weights toward the true label. Repeat for epochs until no mistakes or max epochs.

6. Mathematical Foundation

For labels y in $\{-1, +1\}$:

```
y_hat = sign(w^T x + b)
if y * (w^T x + b) <= 0:
    w := w + eta * y * x
    b := b + eta * y
```

The perceptron converges in finite updates if data is linearly separable.

7. Practical Implementation

```
import numpy as np

class Perceptron:
    def __init__(self, lr=0.1, epochs=20):
        self.lr = lr
        self.epochs = epochs

    def fit(self, X, y):
        self.w = np.zeros(X.shape[1])
        self.b = 0.0
        for _ in range(self.epochs):
            mistakes = 0
            for xi, yi in zip(X, y):
                if yi * (np.dot(self.w, xi) + self.b) <= 0:
                    self.w += self.lr * yi * xi
                    self.b += self.lr * yi
                    mistakes += 1
            if mistakes == 0:
                break

    def predict(self, X):
        return np.where(X @ self.w + self.b >= 0, 1, -1)

X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y = np.array([-1, -1, -1, 1]) # AND
model = Perceptron()
model.fit(X, y)
print(model.predict(X))
```

8. Code Explanation

Weights start at zero. Each misclassified sample changes the boundary. The `predict` method applies the sign rule. The AND dataset is linearly separable, so the perceptron can learn it.

9. Training / Evaluation

Use accuracy for balanced binary data. Standardize features if scales differ. The perceptron may never converge on non-separable data, so use max epochs.

10. Complexity and Cost

Training cost is $O(\text{epochs} * n * d)$. Inference cost is $O(d)$ per sample. Memory is $O(d)$.

11. Common Use Cases

Educational classifier, online linear classification, basis for understanding neural networks and support vector machines.

12. Common Mistakes

Using labels $\{0,1\}$ with the $\{-1,+1\}$ update, expecting probabilities, using it on XOR, forgetting feature scaling, no stopping condition.

13. Edge Cases / Limitations

Fails on non-linearly separable data, sensitive to feature scaling and sample order, does not produce calibrated probabilities.

14. Variations

Averaged perceptron improves stability. Kernel perceptron handles nonlinear boundaries. Logistic regression replaces step activation with sigmoid and optimizes cross entropy.

15. Related Topics

Artificial neuron generalizes perceptron with differentiable activations. SVM also learns a linear separator but maximizes margin. MLP solves XOR using hidden layers.

16. Interview Questions

1. What is a perceptron?
A linear binary classifier with a threshold decision.
2. What is the update rule?
 $w = w + \text{eta} * y * x$ for misclassified samples.
3. When does it converge?
When data is linearly separable.
4. Can it solve XOR?
No.
5. Why is step activation problematic for deep learning?
It is not differentiable.
6. Difference from logistic regression?
Logistic regression outputs probabilities and uses cross entropy.
7. What is a decision boundary?
The hyperplane $w^T x + b = 0$.

8. Why use bias?

To shift the boundary.

9. What is online learning?

Updating after each example.

10. What if data is not separable?

Perceptron can keep making mistakes.

17. Practice Tasks

Train on AND/OR/XOR, plot decision boundary, convert labels correctly, compare perceptron with logistic regression, implement averaged perceptron.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Linear Gate Learner	Learns Boolean gates	NumPy	AND/OR/XOR	Clear fundamentals
Online Spam Toy	Updates per email	Python	SMS spam subset	Online learning
Boundary Visualizer	Animates updates	Streamlit	Synthetic 2D data	Interview demo

19. Quick Revision

Key idea: linear classifier corrected on mistakes. Formula: $\text{sign}(w^T x + b)$. Use for separable binary tasks. Trap: cannot solve XOR. Interview one-liner: perceptron is the simplest trainable linear neuron.

20. Final Cheat Sheet

Definition: binary linear classifier. Input/output: features to class. Steps: score, threshold, update if wrong. Hyperparameters: learning rate, epochs. Metric: accuracy. Pros: simple. Cons: linear only. Best use: learning fundamentals.

MLP

1. Overview

A multilayer perceptron (MLP) is a feed-forward neural network made of fully connected layers and nonlinear activations. It is used for tabular data, embeddings, classification heads, regression, and the feed-forward blocks inside Transformers.

2. Intuition

An MLP stacks many neurons so early layers learn simple patterns and later layers combine them into more complex patterns, like combining edges into shapes and shapes into objects.

3. Prerequisites

Artificial neurons, matrix multiplication, activation functions, loss functions, gradient descent, backpropagation, PyTorch basics.

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Dense layer	Every input connects to every output	General feature mixing	<code>nn.Linear(784,128)</code>	Parameter count
Hidden layer	Intermediate representation	Enables hierarchy	ReLU layer	Depth vs width
Nonlinearity	Activation between linear layers	Prevents collapse to linear model	ReLU	Why activation is needed
Output head	Maps features to task output	Task-specific	logits for classes	Softmax vs logits

5. Algorithm / Working Process

Input passes through linear layer, activation, optional normalization/dropout, repeated hidden layers, and final output layer. Training uses forward pass, loss computation, backpropagation, and optimizer update. Inference uses only forward pass, usually with dropout disabled.

6. Mathematical Foundation

For layer l :

$$z_l = a_{l-1} W_l + b_l$$

$$a_l = f_l(z_l)$$

For classification:

$$\text{logits} = a_L$$

$$P(y=k|x) = \text{softmax}(\text{logits})_k$$

$$L = -\log P(y=\text{true}|x)$$

7. Practical Implementation

```
import torch
import torch.nn as nn

class MLP(nn.Module):
    def __init__(self, in_dim, hidden_dim, num_classes):
        super().__init__()
```

```
self.net = nn.Sequential(
    nn.Linear(in_dim, hidden_dim),
    nn.ReLU(),
    nn.Linear(hidden_dim, hidden_dim),
    nn.ReLU(),
    nn.Linear(hidden_dim, num_classes),
)

def forward(self, x):
    return self.net(x)

model = MLP(in_dim=784, hidden_dim=128, num_classes=10)
x = torch.randn(32, 784)
logits = model(x)
print(logits.shape)
```

8. Code Explanation

The model maps 784-dimensional flattened images to 10 class logits. Hidden layers learn nonlinear representations. The final layer returns raw logits, which should be passed to `nn.CrossEntropyLoss`.

9. Training / Evaluation

Prepare normalized tensors, split train/validation/test, use cross entropy for classification or MSE for regression, monitor accuracy/loss, tune learning rate, hidden size, number of layers, dropout, and weight decay.

10. Complexity and Cost

A dense layer has `input_dim * output_dim + output_dim` parameters. MLPs can become expensive for high-dimensional inputs, which is why CNNs are preferred for images.

11. Common Use Cases

Tabular prediction, classification heads, regression, recommendation systems, feature transformation, Transformer feed-forward networks.

12. Common Mistakes

Applying softmax before `CrossEntropyLoss`, no normalization of inputs, too large MLP for small data, forgetting `model.eval()`, overfitting without validation.

13. Edge Cases / Limitations

Poor spatial inductive bias for images, parameter-heavy, may overfit small datasets, struggles with sequences unless features are engineered.

14. Variations

Deep MLP, residual MLP, MLP-Mixer, gated MLP, dropout MLP, batch-normalized MLP.

15. Related Topics

Perceptron is one layer with threshold. CNNs add spatial structure. Transformers use MLP blocks after attention. Residual connections help train deep MLPs.

16. Interview Questions

1. What is an MLP?
A stack of fully connected layers with nonlinear activations.
2. Why hidden layers?
They learn intermediate representations.
3. Why activation functions?
Without them, the network is linear.
4. What are logits?
Raw unnormalized class scores.
5. How many parameters in `Linear(m,n)`?
 $m*n+n$.
6. Why can MLP overfit?
Many parameters and weak inductive bias.
7. MLP vs CNN?
CNN shares weights and uses locality.
8. MLP for tabular data?
Often useful after preprocessing but tree models can be strong.
9. How train an MLP?
Forward, loss, backward, optimizer step.
10. What improves MLP generalization?
Dropout, weight decay, normalization, more data.

17. Practice Tasks

Build MNIST MLP, tune hidden width, compare ReLU/tanh, add dropout, plot train vs validation loss.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
MNIST MLP	Digit classifier	PyTorch	MNIST	Baseline DL project
Tabular Risk Model	Predicts churn/default	PyTorch, sklearn	Telco churn	Applied ML
Embedding Classifier	Classifies text embeddings	PyTorch	AG News embeddings	Modern AI workflow

19. Quick Revision

Key idea: dense nonlinear function approximator. Formula: $a_l = f(a_{l-1}W_l + b_l)$. Use for tabular, heads, embeddings. Trap: softmax before cross entropy. Interview one-liner: MLPs compose linear maps and

nonlinearities to learn complex functions.

20. Final Cheat Sheet

Definition: feed-forward dense network. Input/output: vector to prediction. Steps: dense, activation, repeat, output. Hyperparameters: depth, width, activation, LR, dropout. Metrics: accuracy, F1, RMSE. Pros: flexible. Cons: data hungry and parameter-heavy. Best use: vector features.

CNN

1. Overview

A Convolutional Neural Network is designed for grid-like data, especially images. It uses convolution filters to detect local patterns such as edges, textures, object parts, and full objects.

CNNs are used in image classification, object detection, segmentation, OCR, medical imaging, satellite analysis, autonomous driving perception, and video understanding.

2. Intuition

Instead of connecting every pixel to every neuron, CNNs scan small filters across an image. A filter may detect vertical edges wherever they appear. Deeper filters combine edges into shapes, then objects.

3. Prerequisites

- Image representation: channels, height, width
- Matrix operations and convolution
- Backpropagation
- Activation functions
- Pooling and padding
- PyTorch tensor shape convention: **NCHW**

4. Core Concepts

Subtopic	Meaning	Why it matters	Simple example	Interview angle
Convolution	Sliding filter dot product	Shares weights and captures local patterns	3x3 edge detector	Why fewer parameters than MLP
Kernel/filter	Learnable local pattern detector	Learns visual features	Sobel-like filter	Kernel size tradeoff
Stride	Step size of filter movement	Controls output size	Stride 2 downsamples	Effect on feature map size
Padding	Border extension	Preserves spatial size	Same padding	Formula for output size

Subtopic	Meaning	Why it matters	Simple example	Interview angle
Pooling	Downsampling operation	Adds translation tolerance	MaxPool 2x2	Max vs average pooling
Channels	Feature maps	Multiple learned patterns	64 filters produce 64 channels	Input/output channel relation

5. Algorithm / Working Process

Input image tensor has shape `[batch, channels, height, width]`. Convolution layers compute feature maps. Activations add non-linearity. Pooling or strided convolution reduces spatial resolution. A classifier head maps final features to class logits. Training uses backpropagation with image augmentations. Inference applies the same forward pass without augmentation except deterministic preprocessing.

6. Mathematical Foundation

2D convolution:

$$Y(i,j,k) = \sum_c \sum_u \sum_v X(i+u,j+v,c) * W(u,v,c,k) + b_k$$

Output size:

$$H_{out} = \text{floor}((H + 2P - K) / S) + 1$$

$$W_{out} = \text{floor}((W + 2P - K) / S) + 1$$

Classification loss is usually cross-entropy.

7. Practical Implementation

```
import torch
from torch import nn

class SmallCNN(nn.Module):
    def __init__(self, classes=10):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(),
            nn.MaxPool2d(2),
        )
        self.classifier = nn.Sequential(
```

```

        nn.AdaptiveAvgPool2d((1, 1)),
        nn.Flatten(),
        nn.Linear(64, classes),
    )

    def forward(self, x):
        return self.classifier(self.features(x))

model = SmallCNN()
x = torch.randn(8, 3, 32, 32)
logits = model(x)
print(logits.shape)

```

8. Code Explanation

`Conv2d` learns image filters. `BatchNorm2d` stabilizes training. `MaxPool2d` reduces resolution.

`AdaptiveAvgPool2d((1,1))` allows the classifier to work with different image sizes. The model outputs raw class logits.

9. Training / Evaluation

Use train/validation/test split by image identity, not by augmented copies. Apply resize, normalization, random crop, flip, color jitter where appropriate. Metrics include accuracy, top-k accuracy, F1, mean IoU for segmentation, and mAP for detection. Tune learning rate, augmentation, batch size, depth, and weight decay.

10. Complexity and Cost

Convolution cost is approximately $O(H * W * C_{in} * C_{out} * K^2)$. CNNs can be efficient because weights are shared. Larger images and many channels increase memory and GPU cost.

11. Common Use Cases

- Image classification
- Face recognition
- Medical image diagnosis
- OCR feature extraction
- Defect detection in manufacturing
- Object detection and segmentation backbones

12. Common Mistakes

- Wrong channel order: `NHWC` instead of `NCHW` in PyTorch
- Not normalizing images
- Leakage from same patient/object across splits
- Too little augmentation
- Flattening too early and creating huge dense layers
- Ignoring class imbalance

13. Edge Cases / Limitations

CNNs have locality bias and may struggle with long-range relationships unless deep or combined with attention. They can be sensitive to domain shift, adversarial perturbations, and poor lighting/camera conditions.

14. Variations

Variation	What changes	When to use	Importance
1D CNN	Convolve over time/text	Signals, text classification	High
3D CNN	Convolve over volume/time	CT scans, video	Medium
Depthwise separable CNN	Splits spatial and channel mixing	Mobile models	High
Dilated CNN	Adds gaps in kernels	Segmentation, larger receptive field	Medium

15. Related Topics

CNN vs MLP: CNN shares weights and preserves spatial structure. CNN vs ViT: CNN has local inductive bias, ViT uses global attention over patches. CNNs are often used inside ResNet and U-Net.

16. Interview Questions

1. Why are CNNs better than MLPs for images?
They use local connectivity, weight sharing, and spatial hierarchy.
2. What is a kernel?
A learnable filter that detects local patterns.
3. What does padding do?
Controls border handling and output size.
4. How does stride affect output?
Larger stride reduces spatial resolution.
5. What is pooling?
Downsampling that summarizes local regions.
6. What is receptive field?
The region of input influencing a feature.
7. Why use batch normalization?
It stabilizes training and often allows faster convergence.
8. What is depthwise separable convolution?
A cheaper convolution used in mobile models.
9. Why avoid large dense layers after conv layers?
They add many parameters and overfit.
10. What is translation equivariance?
A shifted input creates a shifted feature map.

17. Practice Tasks

- Train a CNN on CIFAR-10.
- Calculate output size for different kernel/stride/padding values.

- Compare max pooling and strided convolution.
- Debug an image pipeline with wrong normalization.
- Visualize first-layer filters.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Plant Disease Classifier	Classifies leaf disease	PyTorch, torchvision	PlantVillage	CV project with real impact
Defect Detection	Finds defective products	PyTorch, OpenCV	MVTec AD	Industrial AI relevance
Traffic Sign Classifier	Recognizes road signs	PyTorch	GTSRB	Good autonomous systems project

19. Quick Revision

- Key idea: learn local image filters with shared weights.
- Main formula: output size $\text{floor}((H+2P-K)/S)+1$.
- When to use: image/grid data.
- Important metrics: accuracy, mAP, IoU.
- Common traps: wrong shape, no normalization, leakage.
- Interview one-liner: CNNs exploit spatial locality and weight sharing.

20. Final Cheat Sheet

Item	Answer
Definition	Neural network using convolution filters
Input/output	Image tensor to class/map/features
Main steps	Conv, activation, pooling, head
Key hyperparameters	Kernel, stride, padding, channels, depth
Metrics	Accuracy, F1, IoU, mAP
Pros	Efficient, strong image bias
Cons	Weaker global context than attention
Best use cases	Images, videos, spatial signals

RNN

1. Overview

A Recurrent Neural Network processes sequences by maintaining a hidden state across time steps. It is designed for ordered data such as text, speech, time series, and sensor streams.

RNNs were central in early NLP and speech systems. Today, Transformers dominate many sequence tasks, but RNNs remain useful for small, streaming, low-latency, and embedded sequence models.

2. Intuition

An RNN reads one token at a time while carrying a memory. If the sentence is "I grew up in India, so I speak ...", the hidden state should help predict "Hindi" or another language because previous words matter.

3. Prerequisites

- Sequence data and time steps
- Hidden states
- Matrix multiplication
- Backpropagation through time
- Gradient vanishing/exploding
- Tokenization or time-series preprocessing

4. Core Concepts

Subtopic	Meaning	Why it matters	Simple example	Interview angle
Hidden state	Memory vector	Carries past information	Previous words in sentence	Why RNN handles variable length
Recurrence	Same cell reused over time	Parameter sharing across positions	Same update for each word	Unrolling RNN
BPTT	Backprop through time	Trains recurrent weights	Gradients flow across steps	Vanishing gradients
Many-to-one	Sequence to label	Classification	Sentiment analysis	Which output to use
Many-to-many	Sequence to sequence	Tagging or translation	POS tagging	Alignment issues

5. Algorithm / Working Process

At each time step, the RNN receives input x_t and previous hidden state h_{t-1} . It computes new hidden state h_t . For classification, the final hidden state is passed to a dense layer. For sequence labeling, every hidden state may produce an output.

6. Mathematical Foundation

$$h_t = \tanh(W_{xh} x_t + W_{hh} h_{t-1} + b_h)$$

$$y_t = W_{hy} h_t + b_y$$

Training uses backpropagation through time. The main problem is repeated multiplication by recurrent weights, which can shrink or explode gradients:

dL/dh_t contains product of many W_{hh} terms

7. Practical Implementation

```
import torch
from torch import nn

class RNNClassifier(nn.Module):
    def __init__(self, vocab_size, embed_dim=64, hidden=128, classes=2):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim)
        self.rnn = nn.RNN(embed_dim, hidden, batch_first=True)
        self.fc = nn.Linear(hidden, classes)

    def forward(self, tokens):
        x = self.embedding(tokens)
        output, hidden = self.rnn(x)
        last_hidden = hidden[-1]
        return self.fc(last_hidden)

model = RNNClassifier(vocab_size=5000)
tokens = torch.randint(0, 5000, (16, 30))
logits = model(tokens)
print(logits.shape)
```

8. Code Explanation

`Embedding` converts token IDs into vectors. `nn.RNN` processes the sequence. `hidden[-1]` is the final hidden state from the last recurrent layer. The final linear layer maps sequence representation to class logits.

9. Training / Evaluation

Pad variable-length sequences and use packed sequences if needed. Use cross-entropy for classification or sequence labeling. Monitor gradient norms. Use validation metrics such as accuracy, F1, perplexity, MAE, or RMSE depending on the task.

10. Complexity and Cost

RNN time complexity is $O(T * hidden^2)$ approximately, where T is sequence length. It is sequential across time, so it cannot parallelize as well as Transformers during training.

11. Common Use Cases

- Time-series forecasting

- Sentiment classification
- Speech sequence modeling
- Character-level language modeling
- Embedded streaming models

12. Common Mistakes

- Ignoring padding masks
- Using final hidden state for padded tokens
- Forgetting gradient clipping
- Expecting vanilla RNNs to learn long dependencies
- Mixing batch-first and sequence-first tensor formats

13. Edge Cases / Limitations

Vanilla RNNs struggle with long-term dependencies due to vanishing gradients. Training is slow for long sequences because computation is sequential.

14. Variations

Variation	What changes	When to use	Importance
Bidirectional RNN	Reads forward and backward	Full sequence available	High
Stacked RNN	Multiple recurrent layers	More capacity	Medium
LSTM	Adds gates and cell state	Long dependencies	High
GRU	Simpler gated RNN	Efficient sequence model	High

15. Related Topics

RNN vs LSTM: LSTM adds gates to improve memory. RNN vs Transformer: Transformer parallelizes and uses attention. RNN vs CNN for sequences: CNN sees local windows; RNN maintains state over time.

16. Interview Questions

1. What makes RNNs suitable for sequences?
They maintain hidden state across time.
2. What is BPTT?
Backpropagation applied to the unrolled sequence.
3. Why do RNNs suffer from vanishing gradients?
Gradients are repeatedly multiplied through time.
4. What is hidden state?
A vector summary of previous inputs.
5. What is many-to-one modeling?
Mapping a sequence to one output.
6. What is many-to-many modeling?
Producing output at multiple time steps.

7. Why use gradient clipping?
To prevent exploding gradients.
8. Can RNNs process variable-length sequences?
Yes, with padding/masking or packed sequences.
9. Why are Transformers faster to train?
They parallelize across tokens.
10. When are RNNs still useful?
Streaming, small-data, low-latency sequence tasks.

17. Practice Tasks

- Build a character-level name classifier.
- Train an RNN for stock movement direction.
- Compare RNN, LSTM, and GRU on the same dataset.
- Debug padding-related metric errors.
- Add gradient clipping and compare stability.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Sentiment RNN	Classifies reviews	PyTorch	IMDb	NLP fundamentals
Sensor Fault Predictor	Predicts machine failure	PyTorch, pandas	NASA turbofan	Time-series ML
Character Name Generator	Generates names	PyTorch	Name lists	Sequence generation

19. Quick Revision

- Key idea: hidden state carries sequence memory.
- Main formula: $h_t = \tanh(Wx_t + Uh_{t-1} + b)$.
- When to use: ordered sequence data.
- Important metrics: F1, perplexity, MAE/RMSE.
- Common traps: padding, vanishing gradients.
- Interview one-liner: RNNs reuse the same cell across time to process sequences.

20. Final Cheat Sheet

Item	Answer
Definition	Recurrent sequence neural network
Input/output	Sequence to label/sequence
Main steps	Embed, recurrent update, output head
Key hyperparameters	Hidden size, layers, sequence length, LR
Metrics	Accuracy, F1, perplexity, RMSE

Item	Answer
Pros	Handles variable-length sequences
Cons	Slow and weak on long dependencies
Best use cases	Streaming sequence models

LSTM

1. Overview

Long Short-Term Memory is a gated RNN architecture designed to remember information over longer sequences. It adds a cell state and gates that control what to forget, store, and output.

LSTMs are used in time-series forecasting, speech recognition, handwriting recognition, anomaly detection, and older NLP systems.

2. Intuition

An LSTM is like a notebook with three controls: erase irrelevant notes, write useful new notes, and decide what part of the notes to reveal for the current prediction.

3. Prerequisites

- RNNs and hidden states
- Sigmoid and tanh
- Element-wise multiplication
- Sequence padding/masking
- Backpropagation through time

4. Core Concepts

Subtopic	Meaning	Why it matters	Simple example	Interview angle
Cell state	Long-term memory path	Helps preserve gradients	Topic of paragraph	Difference from hidden state
Forget gate	Decides what to remove	Removes irrelevant past	Forget old subject	Why sigmoid
Input gate	Decides what to write	Stores new useful info	Store negation word	Gate equations
Output gate	Decides what to expose	Controls hidden output	Reveal sentiment info	Hidden vs cell state
Gates	Learnable filters	Dynamic memory control	Values between 0 and 1	LSTM vs vanilla RNN

5. Algorithm / Working Process

For each time step, LSTM combines current input and previous hidden state. It computes forget, input, candidate, and output values. The cell state is updated using forget and input gates. The hidden state is computed from the updated cell state and output gate.

6. Mathematical Foundation

```
f_t = sigmoid(W_f [h_{t-1}, x_t] + b_f)
i_t = sigmoid(W_i [h_{t-1}, x_t] + b_i)
g_t = tanh(W_g [h_{t-1}, x_t] + b_g)
c_t = f_t * c_{t-1} + i_t * g_t
o_t = sigmoid(W_o [h_{t-1}, x_t] + b_o)
h_t = o_t * tanh(c_t)
```

The additive cell update helps gradients flow better than vanilla RNN recurrence.

7. Practical Implementation

```
import torch
from torch import nn

class LSTMClassifier(nn.Module):
    def __init__(self, vocab_size, embed_dim=64, hidden=128, classes=2):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim)
        self.lstm = nn.LSTM(embed_dim, hidden, batch_first=True,
bidirectional=True)
        self.fc = nn.Linear(hidden * 2, classes)

    def forward(self, tokens):
        x = self.embedding(tokens)
        output, (hidden, cell) = self.lstm(x)
        final = torch.cat([hidden[-2], hidden[-1]], dim=1)
        return self.fc(final)

model = LSTMClassifier(8000)
tokens = torch.randint(0, 8000, (16, 40))
print(model(tokens).shape)
```

8. Code Explanation

`nn.LSTM` returns outputs for all time steps plus final hidden and cell states. Because the model is bidirectional, the last two hidden states correspond to forward and backward directions. Concatenating them gives a stronger sequence representation.

9. Training / Evaluation

Use padding masks or packed sequences for variable length data. Apply gradient clipping. Tune hidden size, number of layers, dropout, and learning rate. Evaluate using F1 for classification, RMSE/MAE for forecasting, or word error rate for speech.

10. Complexity and Cost

LSTM has about four times the recurrent computations of vanilla RNN because it has four gate-related transformations. It is still sequential across time.

11. Common Use Cases

- Time-series forecasting
- Sentiment analysis
- Named entity recognition
- Speech and handwriting recognition
- Sequence anomaly detection

12. Common Mistakes

- Confusing cell state and hidden state
- Ignoring bidirectional output shape
- Using LSTM for very long text where Transformer is better
- Not masking padded tokens
- Forgetting gradient clipping

13. Edge Cases / Limitations

LSTMs handle longer dependencies than RNNs but still struggle with very long contexts. Training is slower than attention-based parallel models.

14. Variations

Variation	What changes	When to use	Importance
BiLSTM	Uses future and past context	Tagging/classification	High
Stacked LSTM	More layers	More expressive sequences	Medium
ConvLSTM	Convolution inside gates	Video/weather grids	Medium
Peephole LSTM	Gates see cell state	Fine timing tasks	Low

15. Related Topics

LSTM vs GRU: GRU is simpler and often faster. LSTM vs Transformer: LSTM has recurrence; Transformer uses attention. LSTM is a solution to vanilla RNN gradient issues.

16. Interview Questions

1. Why was LSTM introduced?

To reduce vanishing gradient problems in vanilla RNNs.

2. What are the three main gates?
Forget, input, and output gates.
3. What is the cell state?
A long-term memory vector updated additively.
4. Why does LSTM remember longer dependencies?
Gates control memory and the cell path supports gradient flow.
5. What is a BiLSTM?
An LSTM that reads sequences forward and backward.
6. LSTM vs GRU?
LSTM has separate cell state and more gates; GRU is simpler.
7. Why use sigmoid in gates?
It outputs values between 0 and 1 for soft selection.
8. What does the forget gate do?
It scales old cell state information.
9. Is LSTM parallelizable across time?
Not fully, because each step depends on the previous state.
10. When would you choose LSTM today?
Time series, streaming, or smaller sequence tasks.

17. Practice Tasks

- Train BiLSTM sentiment classifier.
- Forecast temperature using LSTM.
- Compare hidden sizes on validation loss.
- Debug shape errors in bidirectional LSTM.
- Add packed padded sequences.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Energy Forecasting	Predicts electricity demand	PyTorch, pandas	UCI household power	Time-series project
BiLSTM NER	Extracts entities	PyTorch	CoNLL-2003	NLP sequence labeling
Anomaly Detector	Flags abnormal sensor streams	PyTorch	SWaT/NASA	Industrial ML

19. Quick Revision

- Key idea: gated memory with cell state.
- Main formula: $c_t = f_t * c_{t-1} + i_t * g_t$.
- When to use: sequences with medium/long dependencies.
- Important metrics: F1, RMSE, WER.
- Common traps: hidden/cell confusion, padding.
- Interview one-liner: LSTM is an RNN with gates that control long-term memory.

20. Final Cheat Sheet

Item	Answer
Definition	Gated recurrent network
Input/output	Sequence to hidden states/predictions
Main steps	Forget, input, cell update, output
Key hyperparameters	Hidden size, layers, dropout, LR
Metrics	F1, RMSE, WER
Pros	Better long-term memory than RNN
Cons	Slower and heavier than GRU
Best use cases	Forecasting and sequence labeling

GRU

1. Overview

Gated Recurrent Unit is a simpler gated RNN that merges some LSTM ideas into fewer gates. It uses reset and update gates and does not maintain a separate cell state.

GRUs are used for time series, small NLP systems, speech, and real-time sequence processing where LSTM may be heavier than needed.

2. Intuition

A GRU asks two questions at each time step: how much old memory should be kept, and how much past information should be ignored while forming new memory.

3. Prerequisites

- RNN basics
- LSTM intuition
- Sigmoid/tanh
- Hidden states
- Sequence batching and masking

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Update gate	Controls old vs new state	Similar to memory retention	Keep old sentiment context	Compare with LSTM forget/input

Subtopic	Meaning	Why it matters	Example	Interview angle
Reset gate	Controls past usage	Helps ignore irrelevant history	Reset after sentence boundary	Gate interpretation
Candidate state	Proposed new memory	Adds new sequence info	Current word meaning	Formula
No cell state	Hidden state is memory	Simpler than LSTM	One state vector	Speed vs expressiveness

5. Algorithm / Working Process

At each time step, compute reset and update gates. Use reset gate to build candidate memory. Use update gate to interpolate between old hidden state and candidate state.

6. Mathematical Foundation

```

z_t = sigmoid(W_z x_t + U_z h_{t-1})
r_t = sigmoid(W_r x_t + U_r h_{t-1})
h~_t = tanh(W_h x_t + U_h (r_t * h_{t-1}))
h_t = (1 - z_t) * h_{t-1} + z_t * h~_t

```

Some libraries use the opposite convention for z_t , but the idea is interpolation between old and new memory.

7. Practical Implementation

```

import torch
from torch import nn

class GRUForecaster(nn.Module):
    def __init__(self, features=4, hidden=64, horizon=1):
        super().__init__()
        self.gru = nn.GRU(features, hidden, batch_first=True)
        self.head = nn.Linear(hidden, horizon)

    def forward(self, x):
        output, hidden = self.gru(x)
        return self.head(hidden[-1])

model = GRUForecaster()
x = torch.randn(32, 24, 4)
prediction = model(x)
print(prediction.shape)

```

8. Code Explanation

The input shape is `[batch, time, features]`. `nn.GRU` processes the time dimension. `hidden[-1]` is the final sequence summary. The linear head predicts the next value or horizon.

9. Training / Evaluation

For forecasting, create sliding windows from time-series data and split by time, not random rows. Use MAE, RMSE, MAPE, or pinball loss. For classification, use cross-entropy and F1. Tune sequence length, hidden size, learning rate, and batch size.

10. Complexity and Cost

GRU is cheaper than LSTM because it uses fewer gates. Complexity is still sequential in T , roughly $O(T * \text{hidden}^2)$.

11. Common Use Cases

- Demand forecasting
- Sensor prediction
- Text classification
- Speech sequence modeling
- Real-time sequence systems

12. Common Mistakes

- Random splitting time series
- Leakage from future values
- Not scaling continuous features
- Using too long a sequence without need
- Confusing GRU update gate with LSTM output gate

13. Edge Cases / Limitations

GRUs may be less expressive than LSTMs on tasks requiring precise long memory. Like all RNNs, training is not fully parallel across time.

14. Variations

Variation	What changes	When to use	Importance
BiGRU	Reads both directions	Text classification/tagging	High
Stacked GRU	Multiple GRU layers	More capacity	Medium
Attention GRU	Adds attention over states	Better long sequence summaries	Medium

15. Related Topics

GRU vs LSTM: GRU is simpler and faster; LSTM has richer memory control. GRU vs Transformer: GRU is recurrent and streaming-friendly; Transformer is parallel and context-rich.

16. Interview Questions

1. What is a GRU?
A gated RNN with update and reset gates.
2. How is GRU different from LSTM?
GRU has fewer gates and no separate cell state.
3. What does update gate control?
How much new candidate state replaces old hidden state.
4. What does reset gate control?
How much past state influences candidate memory.
5. Why can GRU train faster than LSTM?
It has fewer parameters and computations.
6. Does GRU solve vanishing gradients completely?
No, but it reduces the issue compared with vanilla RNNs.
7. When prefer GRU over LSTM?
When speed and simplicity matter and performance is similar.
8. What tensor shape does PyTorch GRU expect with `batch_first=True`?
`[batch, time, features]`.
9. Can GRU be bidirectional?
Yes, if future context is available.
10. Is GRU good for streaming?
Yes, because it updates state step by step.

17. Practice Tasks

- Forecast daily temperature with GRU.
- Compare GRU vs LSTM validation time and accuracy.
- Build BiGRU text classifier.
- Debug future leakage in forecasting.
- Add teacher forcing for sequence prediction.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Sales Forecast GRU	Predicts sales	PyTorch, pandas	Rossmann/Kaggle	Business forecasting
Wearable Activity Model	Classifies activity	PyTorch	UCI HAR	Sensor ML
News Sentiment GRU	Classifies finance headlines	PyTorch	Financial phrasebank	Finance NLP

19. Quick Revision

- Key idea: simpler gated sequence model.
- Main formula: $h_t = (1 - z_t)h_{t-1} + z_t \tilde{h}_t$.
- When to use: efficient sequence modeling.

- Important metrics: F1, MAE, RMSE.
- Common traps: time leakage, padding.
- Interview one-liner: GRU is a lighter LSTM-style recurrent model.

20. Final Cheat Sheet

Item	Answer
Definition	Gated recurrent unit
Input/output	Sequence to hidden/prediction
Main steps	Reset gate, update gate, candidate state
Key hyperparameters	Hidden size, layers, sequence length
Metrics	F1, MAE, RMSE
Pros	Faster than LSTM
Cons	Less expressive memory than LSTM
Best use cases	Forecasting and real-time sequence tasks

Autoencoder

1. Overview

An Autoencoder is an unsupervised neural network that learns to reconstruct its input through a compressed latent representation. It has an encoder, bottleneck, and decoder.

Autoencoders are used for dimensionality reduction, anomaly detection, denoising, representation learning, compression, and pretraining.

2. Intuition

It is like asking a student to summarize a long paragraph into a few keywords and then reconstruct the paragraph. Good reconstruction means the summary captured important information.

3. Prerequisites

- MLP/CNN basics
- Reconstruction loss
- Latent space
- Optimization
- Normalization

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Encoder	Compresses input	Learns representation	Image to vector	Feature learning
Bottleneck	Low-dimensional latent	Forces compression	784 to 32 dims	Undercomplete AE
Decoder	Reconstructs input	Tests latent usefulness	Vector to image	Reconstruction objective
Reconstruction loss	Difference between input/output	Training signal	MSE/BCE	Which loss to use
Denoising	Reconstruct clean from noisy	Robust features	Noisy image restoration	Denoising AE

5. Algorithm / Working Process

Input is passed through encoder to latent vector z . Decoder maps z back to reconstructed input x_{hat} . Training minimizes reconstruction error between x and x_{hat} . In anomaly detection, high reconstruction error suggests unusual input.

6. Mathematical Foundation

```
z = encoder(x)
x_hat = decoder(z)
L = ||x - x_hat||^2
```

For binary-like pixels:

$$L = -\sum_i [x_i \log(x_{\text{hat}_i}) + (1-x_i)\log(1-x_{\text{hat}_i})]$$

7. Practical Implementation

```
import torch
from torch import nn

class Autoencoder(nn.Module):
    def __init__(self, input_dim=784, latent_dim=32):
        super().__init__()
        self.encoder = nn.Sequential(
            nn.Linear(input_dim, 128),
            nn.ReLU(),
            nn.Linear(128, latent_dim),
        )
        self.decoder = nn.Sequential(
```

```
        nn.Linear(latent_dim, 128),
        nn.ReLU(),
        nn.Linear(128, input_dim),
        nn.Sigmoid(),
    )

    def forward(self, x):
        z = self.encoder(x)
        return self.decoder(z)

model = Autoencoder()
x = torch.rand(16, 784)
x_hat = model(x)
loss = nn.MSELoss()(x_hat, x)
print(loss.item())
```

8. Code Explanation

The encoder reduces 784 input dimensions to 32 latent dimensions. The decoder reconstructs the original vector. `Sigmoid` keeps output in `[0,1]`, suitable for normalized image pixels.

9. Training / Evaluation

Train without labels using reconstruction loss. For anomaly detection, train mostly on normal data and choose a threshold from validation reconstruction errors. Evaluate with reconstruction MSE, anomaly ROC-AUC, precision-recall, or downstream task performance.

10. Complexity and Cost

Cost depends on encoder and decoder size. Dense autoencoders are cheap. Convolutional autoencoders for images cost more but preserve spatial structure better.

11. Common Use Cases

- Image denoising
- Anomaly detection
- Dimensionality reduction
- Feature learning
- Data compression

12. Common Mistakes

- Bottleneck too large, causing identity mapping
- Evaluating only reconstruction visually
- Training anomaly detector with many anomalies
- Wrong output activation for data scale
- Comparing MSE across differently normalized datasets

13. Edge Cases / Limitations

Autoencoders may reconstruct anomalies too well if overpowered. Latent space may not be smooth or generative unless regularized, unlike VAEs.

14. Variations

Variation	What changes	When to use	Importance
Denoising AE	Noisy input, clean target	Robust reconstruction	High
Sparse AE	Sparsity penalty	Interpretable features	Medium
Convolutional AE	CNN encoder/decoder	Images	High
Contractive AE	Penalizes sensitivity	Stable latent space	Low

15. Related Topics

Autoencoder vs PCA: both reduce dimensions; autoencoders learn non-linear mappings. Autoencoder vs VAE: VAE learns probabilistic latent distributions. U-Net is an encoder-decoder with skip connections.

16. Interview Questions

1. What is an autoencoder?
A model trained to reconstruct its input through a latent bottleneck.
2. Why use a bottleneck?
To force useful compressed representation learning.
3. Is it supervised?
Usually unsupervised or self-supervised.
4. What loss is common?
MSE for continuous data, BCE for normalized binary-like data.
5. How use AE for anomaly detection?
High reconstruction error indicates anomaly.
6. AE vs PCA?
AE can learn non-linear compression.
7. What happens if latent dimension is too large?
The model may learn identity mapping.
8. What is denoising AE?
It reconstructs clean input from corrupted input.
9. Is vanilla AE generative?
Not reliably; latent space is not explicitly regularized.
10. Why use convolutional AE for images?
It preserves spatial structure.

17. Practice Tasks

- Build MNIST autoencoder.
- Plot latent vectors with t-SNE.
- Train denoising autoencoder.
- Detect anomalies using reconstruction threshold.
- Compare PCA and autoencoder compression.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Industrial Anomaly AE	Flags defective images	PyTorch	MVTec AD	Strong CV anomaly project
Image Denoiser	Removes noise from images	PyTorch, OpenCV	MNIST/CIFAR	Clear visual demo
Tabular Compression	Learns compact features	PyTorch, sklearn	UCI datasets	Representation learning

19. Quick Revision

- Key idea: reconstruct input through bottleneck.
- Main formula: $L = ||x - \hat{x}||^2$.
- When to use: compression, denoising, anomaly detection.
- Important metrics: reconstruction loss, ROC-AUC.
- Common traps: oversized bottleneck.
- Interview one-liner: an autoencoder learns compressed representations by reconstructing its input.

20. Final Cheat Sheet

Item	Answer
Definition	Encoder-bottleneck-decoder reconstruction model
Input/output	Input to reconstructed input
Main steps	Encode, compress, decode, compare
Key hyperparameters	Latent dim, depth, loss, LR
Metrics	MSE, BCE, anomaly AUC
Pros	Unsupervised representation learning
Cons	Can learn identity mapping
Best use cases	Denoising, compression, anomaly detection

Variational Autoencoder

1. Overview

A Variational Autoencoder is a probabilistic generative autoencoder. Instead of encoding an input to a fixed latent vector, it encodes to a distribution, samples from that distribution, and decodes the sample.

VAEs are used for generative modeling, latent space interpolation, anomaly detection, representation learning, and controllable generation research.

2. Intuition

Vanilla autoencoders learn isolated points in latent space. VAEs shape latent space into a smooth cloud, so sampling nearby points produces meaningful outputs.

3. Prerequisites

- Autoencoders
- Gaussian distributions
- KL divergence
- Reparameterization trick
- Reconstruction loss
- Basic probability

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Latent distribution	Encoder outputs mean and variance	Enables sampling	$\mu, \log\text{var}$	AE vs VAE
Reparameterization	Sampling in differentiable form	Allows backprop	$z = \mu + \sigma * \epsilon$	Why needed
KL divergence	Regularizes latent distribution	Keeps latent near normal	Match $N(0, I)$	ELBO
Reconstruction term	Keeps outputs accurate	Prevents useless latent	MSE/BCE	Balance with KL
Generative sampling	Decode random latent	Creates new data	Sample digit	Why VAE is generative

5. Algorithm / Working Process

Encoder maps input to μ and $\log\text{var}$. Sample ϵ from standard normal. Compute $z = \mu + \sigma * \epsilon$. Decoder reconstructs input from z . Training minimizes reconstruction loss plus KL divergence.

6. Mathematical Foundation

Evidence lower bound:

$$\text{ELBO} = \mathbb{E}_{q(z|x)}[\log p(x|z)] - \text{KL}(q(z|x) || p(z))$$

Loss minimized:

```

L = reconstruction_loss + KL(q(z|x) || N(0,I))
KL = -0.5 * sum(1 + log(sigma^2) - mu^2 - sigma^2)
z = mu + sigma * epsilon, epsilon ~ N(0,I)

```

7. Practical Implementation

```

import torch
from torch import nn
import torch.nn.functional as F

class VAE(nn.Module):
    def __init__(self, input_dim=784, latent_dim=20):
        super().__init__()
        self.fc1 = nn.Linear(input_dim, 400)
        self.mu = nn.Linear(400, latent_dim)
        self.logvar = nn.Linear(400, latent_dim)
        self.fc2 = nn.Linear(latent_dim, 400)
        self.fc3 = nn.Linear(400, input_dim)

    def encode(self, x):
        h = F.relu(self.fc1(x))
        return self.mu(h), self.logvar(h)

    def reparameterize(self, mu, logvar):
        std = torch.exp(0.5 * logvar)
        eps = torch.randn_like(std)
        return mu + std * eps

    def decode(self, z):
        return torch.sigmoid(self.fc3(F.relu(self.fc2(z))))

    def forward(self, x):
        mu, logvar = self.encode(x)
        z = self.reparameterize(mu, logvar)
        return self.decode(z), mu, logvar

    def vae_loss(x_hat, x, mu, logvar):
        recon = F.binary_cross_entropy(x_hat, x, reduction="sum")
        kl = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp())
        return recon + kl

```

8. Code Explanation

The encoder outputs `mu` and `logvar`. The reparameterization trick separates randomness from trainable parameters. The loss combines reconstruction quality and latent regularization.

9. Training / Evaluation

Use normalized inputs. Track reconstruction loss and KL term separately. Evaluate sample quality visually, reconstruction error, latent interpolation, downstream classification, or anomaly detection AUC. Tune latent dimension and KL weight.

10. Complexity and Cost

Similar to autoencoders plus extra mean/logvar layers and sampling. VAEs are cheaper than GANs/diffusion models but often produce blurrier images.

11. Common Use Cases

- Generating simple images
- Latent interpolation
- Anomaly detection
- Semi-supervised learning
- Representation learning

12. Common Mistakes

- Forgetting KL term
- Sampling directly without reparameterization
- Posterior collapse
- Wrong reconstruction loss for data type
- KL term overpowering reconstruction

13. Edge Cases / Limitations

VAEs often produce blurry samples with pixel-wise losses. Posterior collapse can occur, especially with powerful decoders.

14. Variations

Variation	What changes	When to use	Importance
Beta-VAE	Weighted KL term	Disentanglement	Medium
Conditional VAE	Conditions on labels/text	Controlled generation	High
VQ-VAE	Discrete latent codes	Image/audio tokenization	High
Hierarchical VAE	Multiple latent levels	Rich generation	Research

15. Related Topics

VAE vs AE: VAE is probabilistic and generative. VAE vs GAN: VAE optimizes likelihood lower bound; GAN uses adversarial training. VQ-VAE connects to tokenizers for generative models.

16. Interview Questions

1. What is a VAE?

A probabilistic autoencoder trained with reconstruction plus KL loss.

2. Why output mean and variance?
To define a latent distribution.
3. What is the reparameterization trick?
 $z = \mu + \sigma * \epsilon$ to allow backprop through sampling.
4. What is KL divergence doing?
Regularizing latent distribution toward prior.
5. What is ELBO?
A lower bound on log-likelihood optimized by VAE.
6. Why are VAEs generative?
You can sample latent z from prior and decode it.
7. What is posterior collapse?
Decoder ignores latent variable.
8. VAE vs GAN?
VAE has explicit probabilistic objective; GAN learns through discriminator.
9. Why might VAE images be blurry?
Pixel reconstruction losses average possible outputs.
10. What is beta-VAE?
VAE with weighted KL term for disentanglement.

17. Practice Tasks

- Train VAE on MNIST.
- Plot 2D latent space.
- Generate samples from random z .
- Experiment with beta values.
- Detect anomalies with reconstruction plus KL score.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Digit Generator	Samples handwritten digits	PyTorch	MNIST	Generative basics
Fashion Latent Explorer	Interpolates clothing images	PyTorch, Streamlit	Fashion-MNIST	Visual portfolio
VAE Anomaly Detector	Finds abnormal records	PyTorch	Credit fraud/MVTec	Practical anomaly detection

19. Quick Revision

- Key idea: learn smooth probabilistic latent space.
- Main formula: $L = \text{recon} + \text{KL}(q(z|x) || N(0, I))$.
- When to use: generative representation learning.
- Important metrics: recon loss, KL, sample quality.
- Common traps: no reparameterization, posterior collapse.
- Interview one-liner: VAE makes autoencoders generative by encoding distributions, not points.

20. Final Cheat Sheet

Item	Answer
Definition	Probabilistic generative autoencoder
Input/output	Input to reconstruction plus latent distribution
Main steps	Encode mu/logvar, sample, decode, KL+recon loss
Key hyperparameters	Latent dim, beta/KL weight, LR
Metrics	ELBO, recon loss, anomaly AUC
Pros	Smooth latent sampling
Cons	Blurry samples, posterior collapse
Best use cases	Latent generation and anomaly detection

GAN

1. Overview

A Generative Adversarial Network (GAN) trains a generator G to produce samples and a discriminator/critic D to distinguish real from generated data. Their adversarial game can yield sharp, fast samples without an explicit likelihood. GANs remain important for image synthesis, super-resolution, translation, face editing, and interview understanding of distribution matching.

2. Intuition

A counterfeiter improves fake notes while an inspector improves fake detection. Feedback from the inspector teaches the counterfeiter what realism requires. At equilibrium, generated and real distributions match and the inspector cannot do better than chance.

3. Prerequisites

- Neural networks, CNNs, gradient descent, and alternating optimization
- Binary classification and cross-entropy
- Probability distributions, divergences, and expectations
- Image normalization and stable training practices
- PyTorch autograd, `.detach()`, and optimizer state

4. Core Concepts

Concept	Meaning	Why it matters	Simple example	Interview angle
Generator	$G(z)$ maps noise to fake data	Learns implicit distribution	vector $\hat{z}$ ' face	No tractable density

Concept	Meaning	Why it matters	Simple example	Interview angle
Discriminator	Estimates real vs fake	Supplies learned training signal	real score	Optimal discriminator
Minimax game	Opposing objectives	Drives distribution matching	$\min_G \max_D$	Nash equilibrium
Non-saturating loss	Maximizes $\log D(G(z))$	Stronger early gradients	generator BCE to real	Standard practical loss
Mode collapse	Many z map to few outputs	Destroys diversity	same face repeatedly	Diagnosis/mitigation
Critic	Real-valued score in WGAN	Estimates Wasserstein objective	no sigmoid	Lipschitz constraint
Conditional GAN	Feeds class/text condition	Controllable output	generate digit 7	Projection discriminator

5. Algorithm / Working Process

1. Sample a real minibatch x and latent noise z .
2. Generate fake samples $\tilde{x} = G(z)$.
3. Update D to score real high and detached fake low.
4. Resample or reuse noise and update G so $D(G(z))$ scores as real.
5. Repeat alternating steps; optionally maintain an EMA copy of G for inference.
6. At inference, discard D , sample z , and run one generator pass.

Care is required to freeze or detach the correct graph during each update. Discriminator and generator learning rates, update ratio, regularization, and data augmentation strongly affect stability.

6. Mathematical Foundation

Original minimax GAN:

$$\min_G \max_D \mathbb{E}_{x \sim p_{data}} [\log D(x)] + \mathbb{E}_{z \sim p(z)} [\log(1 - D(G(z)))].$$

For fixed G , $D^*(x) = \frac{p_{data}(x)}{p_{data}(x) + p_g(x)}$. Substitution shows the generator minimizes a Jensen-Shannon-divergence-related objective. In practice, the non-saturating generator minimizes

$$\mathcal{L}_G = -\mathbb{E}_z \log D(G(z)),$$

which gives stronger gradients. WGAN uses

$$\min_G \max_{\|D\|_L \leq 1} \mathbb{E}_{x \sim p_{data}} D(x) - \mathbb{E}_z D(G(z)),$$

with a 1-Lipschitz critic, often encouraged by gradient penalty $\lambda(\|\nabla_{\hat{x}} D(\hat{x})\|_2 - 1)^2$.

7. Practical Implementation

```
import torch
from torch import nn

G = nn.Sequential(nn.Linear(32, 128), nn.ReLU(), nn.Linear(128, 784), nn.Tanh())
D = nn.Sequential(nn.Linear(784, 128), nn.LeakyReLU(0.2), nn.Linear(128, 1))
g_opt = torch.optim.Adam(G.parameters(), 2e-4, betas=(0.5, 0.999))
d_opt = torch.optim.Adam(D.parameters(), 2e-4, betas=(0.5, 0.999))

real = torch.rand(64, 784) * 2 - 1
z = torch.randn(64, 32)

# Discriminator update
fake = G(z)
d_loss = nn.functional.softplus(-D(real)).mean() +
nn.functional.softplus(D(fake.detach())).mean()
d_opt.zero_grad(); d_loss.backward(); d_opt.step()

# Generator update (non-saturating logistic loss)
g_loss = nn.functional.softplus(-D(G(torch.randn(64, 32)))).mean()
g_opt.zero_grad(); g_loss.backward(); g_opt.step()
print({"d_loss": d_loss.item(), "g_loss": g_loss.item()})
```

8. Code Explanation

Images are scaled to $[-1, 1]$ to match the generator's `Tanh`. `softplus(-D(real))` and `softplus(D(fake))` are numerically stable logistic losses on raw discriminator logits. `fake.detach()` prevents the discriminator update from changing generator parameters. The generator then receives gradients through a fresh fake batch while trying to raise discriminator scores.

9. Training / Evaluation

Use diverse, deduplicated data and identity/source-aware splits. Monitor sample grids from fixed seeds, FID/KID, precision/recall, discriminator behavior, gradient norms, and latent interpolation. Loss values alone do not correlate reliably with image quality. Improve training with spectral normalization, gradient penalties, balanced update rates, DiffAugment/ADA for limited data, EMA, and established architectures such as StyleGAN.

10. Complexity and Cost

Training requires both networks and alternating backward passes, but generation requires one G forward pass and is usually far faster than diffusion. Memory is dominated by feature maps and optimizer states. High resolution is expensive and sensitive to batch size. There is no sequential token/denoising loop unless the generator architecture itself introduces one.

11. Common Use Cases

- Photorealistic face and avatar synthesis

- Super-resolution and image restoration
- Paired/unpaired image-to-image translation
- Style transfer, attribute editing, and domain adaptation
- Synthetic data and privacy research

12. Common Mistakes

- Forgetting `detach()` during the discriminator update
- Applying sigmoid twice or mixing logits with probability losses
- Judging training from adversarial losses alone
- Letting the discriminator overpower the generator
- Ignoring diversity and reporting only best samples
- Using batch normalization carelessly with tiny batches
- Assuming mode collapse is fixed by simply adding more epochs

13. Edge Cases / Limitations

GAN training can oscillate, diverge, suffer vanishing gradients, or collapse modes. It offers no straightforward normalized likelihood. Performance drops with small or imbalanced data unless regularized carefully. Conditional GANs can ignore conditions. Generated samples may memorize identities, and training stability is more sensitive than for many reconstruction or diffusion objectives.

14. Variations

Variation	What changes	When to use	Importance
DCGAN	Convolutional architectural rules	Educational image generation	Placement basic
cGAN	Adds labels/conditions	Controlled synthesis	Essential
WGAN-GP	Wasserstein critic + gradient penalty	More meaningful gradients	Essential interview
Pix2Pix	Paired conditional GAN + L1	Paired translation	Project important
CycleGAN	Cycle consistency, unpaired domains	Unpaired translation	Common interview
StyleGAN	Style-modulated synthesis	High-quality faces/assets	Advanced/project

15. Related Topics

- **GAN vs VAE:** GANs favor sharp implicit samples; VAEs optimize an explicit variational likelihood bound.
- **GAN vs diffusion:** GANs have fast one-pass inference; diffusion usually offers easier training and better coverage.
- **Discriminator vs classifier:** a discriminator's negative class evolves as the generator changes.
- **JS vs Wasserstein:** Wasserstein distance can provide useful gradients when supports do not overlap.
- **Pix2Pix vs CycleGAN:** Pix2Pix needs aligned pairs; CycleGAN uses unpaired sets and cycle consistency.

16. Interview Questions

1. **What is the GAN objective?** A minimax game in which D separates real/fake and G makes generated samples indistinguishable.

2. **Why use non-saturating generator loss?** The original minimax generator can have vanishing gradients when D confidently rejects early fakes.
3. **What is mode collapse?** Distinct latent inputs map to too few output modes, producing low diversity.
4. **How do you detect collapse?** Repeated samples, poor recall, low latent sensitivity, class-coverage tests, and nearest-neighbor analysis.
5. **Why does WGAN help?** Wasserstein distance changes smoothly as distributions move and can supply informative critic gradients.
6. **Why enforce Lipschitz continuity?** Kantorovich–Rubinstein duality requires a 1-Lipschitz critic for the Wasserstein objective.
7. **What does `detach()` do?** Stops autograd from propagating the discriminator loss into G during the D update.
8. **Why can't loss alone select the best GAN?** Adversarial losses depend on both changing players and do not directly measure perceptual quality or coverage.
9. **What is feature matching?** Train G to match intermediate discriminator feature statistics, which can stabilize and improve coverage.
10. **How is a conditional GAN built?** Inject condition into G and D , through concatenation, conditional normalization, or projection.
11. **What happens at ideal equilibrium?** $p_g = p_{data}$ and the optimal discriminator returns $1/2$ everywhere.
12. **Why are GANs still useful?** Their one-pass generators offer excellent latency and mature specialized image translation/super-resolution models.

17. Practice Tasks

- **Coding:** train a DCGAN on Fashion-MNIST with fixed-seed sample grids.
- **Dataset project:** build a conditional GAN for balanced class augmentation.
- **Experiment:** compare BCE GAN and WGAN-GP stability and coverage.
- **Debugging:** find graph leakage caused by missing `detach()`.
- **Extension:** add spectral normalization and EMA; quantify the change in FID.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Product recoloring	Translates product styles	Pix2Pix, PyTorch	Facades/custom paired data	Conditional translation
Rare-class augments	Synthesizes minority examples	cGAN, scikit-learn	Imbalanced image set	Links generation to outcomes
GAN stability lab	Compares objectives/regularizers	PyTorch, W&B/MLflow	CIFAR-10	Research experimentation

19. Quick Revision

- **Key idea:** generator learns through an adversarially trained realism signal.
- **Main formula:** $\min_G \max_D E \log D(x) + E \log(1 - D(G(z)))$.
- **When to use:** sharp images and very fast generation/translation.
- **Metrics:** FID/KID, precision/recall, diversity, task/human scores.

- **Common traps:** collapse, unstable balance, detach/logit errors, cherry-picking.
- **Interview one-liner:** “A GAN matches distributions through a two-player game rather than an explicit likelihood.”

20. Final Cheat Sheet

Item	Summary
Definition	Adversarial generator-discriminator framework
Input/output	Noise/condition $\hat{z}$ generated sample
Main steps	Generate fake $\hat{z}$ train D $\hat{z}$ train G $\hat{z}$ alternate
Key hyperparameters	LR, betas, update ratio, latent size, regularization
Metrics	FID/KID, precision/recall, diversity, latency
Pros	Sharp samples, one-pass fast inference
Cons	Unstable, mode collapse, no tractable likelihood
Best use cases	Image synthesis, translation, super-resolution

DCGAN

1. Overview

A Deep Convolutional GAN (DCGAN) is a GAN whose generator and discriminator use convolutional design principles. The generator upsamples a latent vector into an image with transposed convolutions; the discriminator downsamples an image with strided convolutions. DCGAN made GAN training more stable and showed that unsupervised convolutional features can organize meaningful visual concepts.

It is used for image synthesis, representation learning, data augmentation, anomaly detection, and as a teaching baseline for modern generative vision systems.

2. Intuition

The generator is a learned image decoder: it starts from a small random code and repeatedly increases spatial resolution. The discriminator is a learned image critic: it compresses an image and decides whether it resembles the training distribution. Like a counterfeiter and investigator improving together, each network supplies the other with increasingly difficult examples.

3. Prerequisites

- GAN minimax training, binary cross-entropy, and alternating optimization
- Convolution, stride, padding, transposed convolution, and receptive fields
- Batch normalization, ReLU, LeakyReLU, and weight initialization
- PyTorch modules, dataloaders, image normalization, and GPU training
- Basic probability: latent distributions and data distributions

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Latent vector	Random code $z \sim N(0, I)$	Provides controllable variation	100-dimensional vector	Why interpolate in latent space?
Generator	Maps z to an image	Learns the data distribution implicitly	$100 \times 1 \times 1 \rightarrow 3 \times 64 \times 64$	Why use transposed convolution?
Discriminator	Scores real versus generated images	Supplies the learning signal	$3 \times 64 \times 64 \rightarrow \text{scalar logit}$	Why omit pooling layers?
Strided convolution	Learns downsampling	Better task-specific reduction than fixed pooling	Stride 2 halves resolution	Checkerboard/artifact trade-offs
BatchNorm	Stabilizes intermediate statistics	Improves gradient flow	Used in most hidden layers	Where should it be omitted?
Mode collapse	Many z values produce similar images	Destroys sample diversity	Same face with minor changes	Detection and remedies

5. Algorithm / Working Process

1. Normalize real images to $[-1, 1]$ to match the generator's $\tanh$ output.
2. Sample z with shape $[B, \text{latent_dim}, 1, 1]$.
3. Generate fake images $x_{\text{fake}} = G(z)$ through upsampling blocks.
4. Update D using real images labeled real and detached fake images labeled fake.
5. Resample or reuse noise; update G so $D(G(z))$ is classified as real.
6. Repeat alternating updates. Save fixed-noise samples to judge progress consistently.
7. At inference, discard D , sample latent vectors, and run G in evaluation mode.

6. Mathematical Foundation

The original adversarial objective is

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))]$$

In practice the non-saturating generator loss gives stronger early gradients:

```
L_D = BCEWithLogits(D(x_real), 1) + BCEWithLogits(D(G(z).detach()), 0)
L_G = BCEWithLogits(D(G(z)), 1)
```

For a transposed convolution, an output dimension is

$$H_{out} = (H_{in} - 1)s - 2p + k + output_padding$$

DCGAN commonly initializes convolution weights from $N(0, 0.02^2)$. The original recipe uses Adam with a relatively small first-moment coefficient, often `beta1=0.5`, to reduce oscillation.

7. Practical Implementation

```
import torch
from torch import nn

class Generator(nn.Module):
    def __init__(self, z_dim=100, channels=3, width=64):
        super().__init__()
        self.net = nn.Sequential(
            nn.ConvTranspose2d(z_dim, width * 8, 4, 1, 0, bias=False),
            nn.BatchNorm2d(width * 8), nn.ReLU(True),          # 4x4
            nn.ConvTranspose2d(width * 8, width * 4, 4, 2, 1, bias=False),
            nn.BatchNorm2d(width * 4), nn.ReLU(True),          # 8x8
            nn.ConvTranspose2d(width * 4, width * 2, 4, 2, 1, bias=False),
            nn.BatchNorm2d(width * 2), nn.ReLU(True),          # 16x16
            nn.ConvTranspose2d(width * 2, width, 4, 2, 1, bias=False),
            nn.BatchNorm2d(width), nn.ReLU(True),              # 32x32
            nn.ConvTranspose2d(width, channels, 4, 2, 1, bias=False),
            nn.Tanh(),                                         # 64x64
        )
    def forward(self, z):
        return self.net(z)

class Discriminator(nn.Module):
    def __init__(self, channels=3, width=64):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv2d(channels, width, 4, 2, 1, bias=False),
            nn.LeakyReLU(0.2, inplace=True),
            nn.Conv2d(width, width * 2, 4, 2, 1, bias=False),
            nn.BatchNorm2d(width * 2), nn.LeakyReLU(0.2, True),
            nn.Conv2d(width * 2, width * 4, 4, 2, 1, bias=False),
            nn.BatchNorm2d(width * 4), nn.LeakyReLU(0.2, True),
            nn.Conv2d(width * 4, width * 8, 4, 2, 1, bias=False),
            nn.BatchNorm2d(width * 8), nn.LeakyReLU(0.2, True),
            nn.Conv2d(width * 8, 1, 4, 1, 0, bias=False),      # raw logit
        )
    def forward(self, x):
```

```

        return self.net(x).flatten()

device = "cuda" if torch.cuda.is_available() else "cpu"
G, D = Generator().to(device), Discriminator().to(device)
loss_fn = nn.BCEWithLogitsLoss()
opt_g = torch.optim.Adam(G.parameters(), lr=2e-4, betas=(0.5, 0.999))
opt_d = torch.optim.Adam(D.parameters(), lr=2e-4, betas=(0.5, 0.999))

# One training step; real must be a normalized [B, 3, 64, 64] batch.
def train_step(real):
    real = real.to(device)
    z = torch.randn(real.size(0), 100, 1, 1, device=device)
    fake = G(z)

    opt_d.zero_grad()
    d_loss = (loss_fn(D(real), torch.ones(real.size(0), device=device)) +
              loss_fn(D(fake.detach()), torch.zeros(real.size(0), device=device)))
    d_loss.backward(); opt_d.step()

    opt_g.zero_grad()
    g_loss = loss_fn(D(fake), torch.ones(real.size(0), device=device))
    g_loss.backward(); opt_g.step()
    return d_loss.item(), g_loss.item()

```

8. Code Explanation

The generator doubles spatial resolution at each stride-2 block. The discriminator mirrors this reduction and returns raw logits because `BCEWithLogitsLoss` applies the stable sigmoid-plus-BCE computation.

`fake.detach()` prevents the discriminator update from changing `G`; the generator update deliberately does not detach. The first discriminator layer and final generator/discriminator layers omit BatchNorm, following the stable DCGAN recipe.

9. Training / Evaluation

Use a consistent crop/resize and normalize every channel with mean and standard deviation `0.5`. Track fixed-noise image grids, discriminator scores, and diversity—not losses alone. Evaluate with FID (distribution quality), KID (small-sample alternative), precision/recall for fidelity/diversity, and human inspection. Tune learning rates, `beta1`, channel width, latent dimension, discriminator-to-generator update ratio, and augmentation. A held-out real set is needed for honest FID.

10. Complexity and Cost

Convolutional cost is approximately $O(HW * C_{in} * C_{out} * k^2)$ per layer. Training stores activations and optimizer state for two networks and needs both forward/backward paths, while inference needs only `G`. A 64x64 DCGAN is practical on one modest GPU; high resolution becomes unstable and memory-intensive.

11. Common Use Cases

- Synthetic face, product, artwork, and texture generation
- Unsupervised feature learning and latent interpolation

- Rare-class augmentation after careful bias checks
- GAN-based anomaly detection
- Baseline for generative-model research and coursework

12. Common Mistakes

- Feeding `[0,1]` images to a generator ending in `tanh`
- Applying sigmoid in `D` and also using `BCEWithLogitsLoss`
- Forgetting `detach()` during the discriminator update
- Judging quality only from GAN loss values
- Training `D` until generator gradients vanish
- Using tiny or non-representative FID sample sets
- Forgetting `G.eval()` when BatchNorm statistics should be fixed

13. Edge Cases / Limitations

DCGAN may collapse to a few modes, oscillate, memorize small datasets, and produce checkerboard artifacts. It has no likelihood for direct density evaluation. The fixed convolutional hierarchy is weak for very high-resolution or globally coherent scenes, and synthetic data can reproduce demographic or copyright-related biases.

14. Variations

Variation	Change	When to use	Importance
WGAN-GP	Wasserstein critic plus gradient penalty	More stable optimization	High interview/research
Conditional DCGAN	Concatenate labels/embeddings	Class-controlled synthesis	High project value
Spectral-normalized GAN	Constrain discriminator Lipschitz behavior	Stabilize <code>D</code>	Medium-high
Progressive GAN	Grow resolution over training	Higher-resolution images	Research context

15. Related Topics

DCGAN replaces the MLPs of a vanilla GAN with convolutional inductive bias. VAEs optimize a tractable lower bound but often yield blurrier images. Diffusion models train more stably and dominate many high-fidelity tasks, but require many denoising steps. CycleGAN adds two generators and cycle consistency for unpaired translation.

16. Interview Questions

1. **What makes a GAN a DCGAN?** Convolutional generator/discriminator design with strided downsampling, learned upsampling, normalization, and prescribed activations.
2. **Why `tanh` at the generator output?** It bounds pixels to `[-1,1]`, matching normalized training images.
3. **Why LeakyReLU in `D`?** It preserves a gradient for negative activations.

4. **Why no pooling?** Strided convolutions learn the downsampling operation.
5. **Why use logits?** `BCEWithLogitsLoss` is more numerically stable than sigmoid followed by BCE.
6. **Why detach fake images for D?** To avoid computing/updating generator gradients during the discriminator step.
7. **What is mode collapse?** Many latent codes map to too few output modes.
8. **Why can losses be misleading?** GAN training is a game; lower loss does not map monotonically to perceptual quality.
9. **How do you detect memorization?** Nearest-neighbor checks against training images and held-out distribution metrics.
10. **How would you stabilize training?** Balance learning rates, use spectral norm or gradient penalties, suitable initialization, and monitor both fidelity and diversity.
11. **DCGAN versus VAE?** DCGAN often produces sharper samples; VAE has explicit probabilistic inference and a meaningful encoder.

17. Practice Tasks

- Implement the discriminator step and verify generator gradients remain `None`.
- Train on CIFAR-10 or a single CelebA category and graph FID over epochs.
- Compare transposed convolution with nearest-neighbor upsampling plus convolution.
- Diagnose a run where every latent vector generates nearly the same image.
- Extend the model into a class-conditional DCGAN.

18. Project Ideas

Project	What it does	Stack	Dataset	Resume value
Sneaker generator	Synthesizes and interpolates footwear	PyTorch, Gradio	UT Zappos50K	End-to-end generative demo
Defect anomaly detector	Flags images poorly reconstructed/inverted by GAN	PyTorch, OpenCV	MVTec AD	Industrial CV relevance
Conditional art studio	Generates images by category	PyTorch, FastAPI	WikiArt subset	Conditional modeling/deployment

19. Quick Revision

- **Key idea:** convolutional adversarial image generator and discriminator.
- **Main formula:** $\min_G \max_D \mathbb{E} \log D(x) + \mathbb{E} \log(1 - D(G(z)))$.
- **Use:** small/medium-resolution image synthesis and GAN baselines.
- **Metrics:** FID, KID, precision/recall, diversity, human inspection.
- **Traps:** normalization mismatch, double sigmoid, missing detach, mode collapse.
- **One-liner:** “DCGAN stabilizes vanilla GAN image modeling with a convolutional architecture and disciplined normalization/activation choices.”

20. Final Cheat Sheet

Item	Summary
Definition	Convolutional GAN for image generation
Input/output	Random latent tensor -> synthetic image
Steps	Generate; train D on real/fake; train G to fool D
Hyperparameters	z_dim , channels, learning rates, betas, batch size, update ratio
Metrics	FID, KID, precision/recall, visual diversity
Pros	Sharp samples, fast one-pass generation, interpretable latent interpolation
Cons	Unstable game, mode collapse, no explicit likelihood
Best use	Educational baselines and modest-resolution synthesis

CycleGAN

1. Overview

CycleGAN performs image-to-image translation between two visual domains without paired examples. It learns **G**: $X \rightarrow Y$ and **F**: $Y \rightarrow X$, plus one discriminator per domain. Adversarial losses make outputs look like the target domain; cycle-consistency and identity losses preserve source content.

Typical uses include season transfer, artistic style conversion, simulation-to-real adaptation, stain normalization, and appearance translation when aligned source-target pairs are unavailable.

2. Intuition

Suppose you have unrelated collections of horse and zebra photographs. A normal supervised translator needs the same scene once as a horse and once as a zebra. CycleGAN only needs two collections. If **G** changes a horse into a zebra, **F** should reconstruct the original horse; this round trip discourages arbitrary outputs.

3. Prerequisites

- GAN training and convolutional encoder-decoder networks
- Residual blocks, instance normalization, and PatchGAN discriminators
- Paired versus unpaired datasets and domain shift
- L1 loss, adversarial loss, and multi-objective optimization
- PyTorch training loops and image augmentation

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
Two mappings	G : $X \rightarrow Y$, F : $Y \rightarrow X$	Supports both translation directions	Horse → zebra	Why not one generator?

Concept	Meaning	Why it matters	Example	Interview angle
Two discriminators	D_Y and D_X	Enforce realism in each domain	Zebra critic, horse critic	Independent objectives
Cycle consistency	Round trip reconstructs input	Preserves content without pairs	$F(G(x)) \approx x$	Is it sufficient for semantics?
Identity loss	Target-domain input should stay similar	Preserves color/composition	$G(y) \approx y$	When is it useful?
PatchGAN	Classifies local patches	Models texture efficiently	70x70 patch scores	Patch versus image discriminator
Replay buffer	Reuses older fake samples	Reduces discriminator oscillation	Queue of generated images	Stability mechanism

5. Algorithm / Working Process

1. Sample an unpaired mini-batch x from domain X and y from domain Y .
2. Produce $fake_y = G(x)$ and $fake_x = F(y)$.
3. Reconstruct $cycle_x = F(fake_y)$ and $cycle_y = G(fake_x)$.
4. Optionally compute identities $G(y)$ and $F(x)$.
5. Update both generators using adversarial, cycle, and identity losses.
6. Update D_Y using real y and detached/replayed $fake_y$; update D_X analogously.
7. At inference, retain only the generator for the desired direction.

6. Mathematical Foundation

```

L_GAN(G,D_Y) = E_y[(D_Y(y)-1)^2] + E_x[D_Y(G(x))^2]
L_cyc = E_x ||F(G(x))-x||_1 + E_y ||G(F(y))-y||_1
L_id = E_y ||G(y)-y||_1 + E_x ||F(x)-x||_1
L_total = L_GAN(G,D_Y) + L_GAN(F,D_X)
          + lambda_cyc L_cyc + lambda_id L_id

```

Least-squares GAN loss is common because it supplies smoother gradients. λ_{cyc} controls faithfulness versus stylistic freedom; λ_{id} discourages needless changes. Cycle consistency constrains invertibility, but does not mathematically guarantee semantic correctness.

7. Practical Implementation

```

import torch
from torch import nn

l1 = nn.L1Loss()

```

```

mse = nn.MSELoss()

def generator_loss(G, F, D_x, D_y, x, y, lambda_cyc=10.0, lambda_id=5.0):
    fake_y, fake_x = G(x), F(y)
    cycle_x, cycle_y = F(fake_y), G(fake_x)

    adv = (mse(D_y(fake_y), torch.ones_like(D_y(fake_y))) +
           mse(D_x(fake_x), torch.ones_like(D_x(fake_x))))
    cycle = l1(cycle_x, x) + l1(cycle_y, y)
    identity = l1(G(y), y) + l1(F(x), x)
    total = adv + lambda_cyc * cycle + lambda_id * identity
    return total, fake_x, fake_y

def discriminator_loss(D, real, fake):
    real_loss = mse(D(real), torch.ones_like(D(real)))
    fake_loss = mse(D(fake.detach()), torch.zeros_like(D(fake.detach())))
    return 0.5 * (real_loss + fake_loss)

# G/F are residual encoder-decoders; D_x/D_y are PatchGAN critics.
# gen_opt.zero_grad(); loss_g, fake_x, fake_y = generator_loss(...)
# loss_g.backward(); gen_opt.step()
# Then update D_x and D_y separately with discriminator_loss.

```

8. Code Explanation

The snippet isolates the distinctive CycleGAN logic from ordinary CNN boilerplate. Generator loss computes both directions, both round trips, and identity preservation. `ones_like` matches a PatchGAN score map rather than assuming one scalar. Fake samples are detached for discriminator updates so generator parameters are not changed by the wrong optimizer.

9. Training / Evaluation

Create independent domain loaders; paired ordering is unnecessary. Apply geometrically compatible augmentations, normalize to `[-1,1]`, and prevent validation/test leakage by entity or scene. Evaluate target realism with FID/KID, content preservation with LPIPS/SSIM when meaningful, task-specific downstream scores, and human preference. Monitor identity/cycle error, but never treat low cycle loss as proof of correct translation. Tune learning rate, residual depth, `lambda_cyc`, `lambda_id`, patch size, and replay-buffer size.

10. Complexity and Cost

Training maintains four networks and executes multiple generator passes (`G(x)`, `F(y)`, `F(G(x))`, `G(F(y))`), so it is substantially costlier than a one-way GAN. Memory scales with image resolution and stored activations. Inference is a single fully convolutional generator and is much cheaper.

11. Common Use Cases

- Day→night, summer→winter, photo→painting translation
- Synthetic-to-real domain adaptation
- Medical stain/modality appearance normalization with expert validation
- Product recoloring and texture transfer

- Data generation where paired acquisition is expensive

12. Common Mistakes

- Accidentally pairing or shuffling domains in a biased way
- Omitting cycle loss and expecting source content to survive
- Using identity loss when large geometry changes are required
- Calling visually plausible output semantically correct
- Evaluating on training images or only with pixel metrics
- Letting one discriminator overpower its generator
- Forgetting fake-image detach or replay buffering

13. Edge Cases / Limitations

CycleGAN struggles with large geometric changes, multimodal mappings, rare structures, and domains with different information content. It can hide information in imperceptible signals to satisfy reconstruction, hallucinate clinically important features, or learn an unintended shortcut. Unpaired translation is therefore unsafe for high-stakes use without domain-specific validation.

14. Variations

Variation	Change	When to use	Importance
CUT	Contrastive patch loss; one-sided mapping	Faster, lighter unpaired translation	High project/research
UNIT/MUNIT	Shared or multimodal latent spaces	Multiple valid outputs	Medium-high
StarGAN	One model handles many domains	Multi-attribute translation	High interview
Cycle-consistent diffusion	Diffusion-based translation	Higher fidelity/stability	Research

15. Related Topics

Pix2Pix uses paired images and a conditional GAN, usually giving stronger alignment. DCGAN is unconditional generation rather than translation. U-Net is often the generator backbone for paired translation, while the original CycleGAN commonly uses residual blocks. Domain adaptation uses similar objectives but may optimize downstream task performance rather than pixels.

16. Interview Questions

1. **Why is CycleGAN called unpaired?** Corresponding source and target images are not required.
2. **Why two generators?** The reverse mapping supplies the round-trip constraint.
3. **What prevents G from outputting any target-looking image?** Cycle consistency penalizes loss of reconstructable source information.
4. **What does PatchGAN predict?** A grid of local real/fake scores.
5. **Why L1 for cycle loss?** It is robust and generally blurs less than L2.

6. **What is identity loss for?** Preserving already-correct target images, colors, and composition.
7. **Can low cycle loss guarantee semantic preservation?** No; information hiding and shortcut mappings are possible.
8. **CycleGAN versus Pix2Pix?** CycleGAN uses unpaired domains and two directions; Pix2Pix uses aligned pairs.
9. **Why use a fake-image buffer?** Older fakes reduce rapid discriminator overfitting to the newest generator.
10. **How do you evaluate it?** Combine realism, content/task preservation, diversity, and human/domain-expert review.
11. **When should CycleGAN not be used?** When hallucination risk is unacceptable or paired supervision is readily available.

17. Practice Tasks

- Implement cycle and identity loss with shape assertions.
- Train Monet→photo translation and compare with/without identity loss.
- Sweep `lambda_cyc` and measure realism-content trade-offs.
- Debug a model that changes background geometry instead of texture.
- Replace cycle loss with a contrastive feature-preservation experiment.

18. Project Ideas

Project	What it does	Stack	Dataset	Resume value
Weather translator	Converts clear-road scenes to rain/fog	PyTorch, OpenCV	BDD100K subsets	Domain adaptation story
Artwork restyler	Photo→art translation with controls	PyTorch, Gradio	Monet2Photo/WikiArt	Deployable visual demo
Synthetic-to-real adapter	Adapts rendered objects for a classifier	PyTorch, torchvision	VisDA-2017	Measures downstream impact

19. Quick Revision

- **Key idea:** adversarial realism plus reversible, unpaired translation.
- **Main formula:** adversarial losses + `lambda_cyc * ||F(G(x)) - x||_1` in both directions.
- **Use:** when two domains exist but aligned examples do not.
- **Metrics:** FID/KID, LPIPS/SSIM where appropriate, task score, expert review.
- **Traps:** semantic hallucination, excessive identity loss, misleading pixel metrics.
- **One-liner:** “CycleGAN constrains two adversarial mappings with round-trip reconstruction so unpaired domains can supervise each other.”

20. Final Cheat Sheet

Item	Summary
Definition	Unpaired bidirectional image-translation GAN
Input/output	Image in domain X -> translated image in Y (and reverse)
Steps	Translate, cycle back, enforce target realism and reconstruction
Hyperparameters	<code>lambda_cyc</code> , <code>lambda_id</code> , learning rate, patch size, residual blocks
Metrics	FID/KID, content/task preservation, human evaluation
Pros	No aligned pairs; learns both directions
Cons	Four networks, unstable training, semantic/hallucination risk
Best use	Appearance/style translation with unpaired domain collections

Transformer

1. Overview

The Transformer is a neural architecture based on self-attention, feedforward layers, residual connections, normalization, and positional information. It is the foundation of modern LLMs, BERT-style encoders, translation systems, Vision Transformers, speech models, and multimodal AI.

2. Intuition

A Transformer lets every token talk to every other token in parallel. Instead of reading a sentence word by word like an RNN, it builds contextual meaning by comparing all tokens at once.

3. Prerequisites

- Attention and Q/K/V
- Embeddings
- Positional encoding
- LayerNorm
- Residual connections
- Cross-entropy and language modeling

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Self-attention	Token-token interaction	Captures context	"bank" depends on nearby words	Formula
Multi-head attention	Multiple attention views	Learns different relations	syntax + coreference	Why heads

Subtopic	Meaning	Why it matters	Example	Interview angle
Positional encoding	Adds order info	Attention alone is permutation-invariant	token index	Sinusoidal vs learned
Feedforward block	Per-token MLP	Adds non-linear transformation	expand 768 to 3072	Why same for each token
Residual connection	Adds input to output	Helps gradients	$x + \text{block}(x)$	Deep training
LayerNorm	Normalizes features	Stabilizes training	pre-norm Transformer	BatchNorm vs LayerNorm

5. Algorithm / Working Process

Input tokens become embeddings. Positional information is added. Each Transformer block applies self-attention, residual connection, normalization, feedforward network, another residual connection, and normalization. Encoder-only models output contextual embeddings. Decoder-only models predict next tokens using causal masks. Encoder-decoder models use cross-attention for conditional generation.

6. Mathematical Foundation

Attention:

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k})V$$

Transformer block, simplified:

```
x = x + MHA(LayerNorm(x))
x = x + FFN(LayerNorm(x))
```

Language modeling loss:

$$L = -\sum_t \log P(x_t \mid x_{<t})$$

7. Practical Implementation

```
import torch
from torch import nn

class TinyTransformerClassifier(nn.Module):
    def __init__(self, vocab_size, classes=2, d_model=128, heads=4, layers=2):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, d_model)
```

```

self.pos = nn.Parameter(torch.randn(1, 128, d_model))
block = nn.TransformerEncoderLayer(
    d_model=d_model,
    nhead=heads,
    dim_feedforward=4 * d_model,
    batch_first=True,
)
self.encoder = nn.TransformerEncoder(block, num_layers=layers)
self.head = nn.Linear(d_model, classes)

def forward(self, tokens):
    x = self.embedding(tokens) + self.pos[:, :tokens.size(1)]
    x = self.encoder(x)
    return self.head(x[:, 0])

model = TinyTransformerClassifier(vocab_size=10000)
tokens = torch.randint(0, 10000, (8, 64))
print(model(tokens).shape)

```

8. Code Explanation

The embedding layer maps token IDs to vectors. The learned positional parameter gives the model order information. `TransformerEncoderLayer` contains multi-head attention and feedforward layers. The first token representation is used as a classification summary.

9. Training / Evaluation

For classification, use labeled text and cross-entropy. For language modeling, use next-token prediction and perplexity. Use attention masks for padding and causal masks for generation. Important hyperparameters are model dimension, heads, layers, context length, learning rate schedule, warmup, and weight decay.

10. Complexity and Cost

Self-attention cost is $O(n^2 d)$ for sequence length n and dimension d . Feedforward cost is often large too: $O(n d d_{ff})$. Transformers usually require GPUs/TPUs for large-scale training.

11. Common Use Cases

- LLMs and chatbots
- Translation and summarization
- Semantic search embeddings
- Code generation
- Vision Transformers
- Speech and multimodal models

12. Common Mistakes

- Forgetting positional encoding
- Missing causal mask in decoder
- Confusing encoder-only, decoder-only, and encoder-decoder

- Using too long context without memory planning
- Fine-tuning large models with too high learning rate

13. Edge Cases / Limitations

Transformers are data-hungry and expensive. Standard attention scales poorly with long sequences. They can hallucinate and may learn shortcuts from biased data.

14. Variations

Variation	What changes	When to use	Importance
Encoder-only	Bidirectional attention	Classification, embeddings	High
Decoder-only	Causal attention	LLM generation	High
Encoder-decoder	Cross-attention	Translation/summarization	High
Sparse Transformer	Reduced attention pattern	Long context	Medium
MoE Transformer	Sparse expert FFNs	Huge models efficiently	High

15. Related Topics

Transformer vs RNN: Transformers parallelize and model long context better. Transformer vs CNN: less local bias, more global interaction. Transformer vs Mamba: Transformers use attention; Mamba uses state space sequence modeling.

16. Interview Questions

1. What is a Transformer?
A neural architecture built mainly on self-attention and feedforward blocks.
2. Why are positional encodings needed?
Self-attention alone has no order awareness.
3. What is multi-head attention?
Attention run in several subspaces.
4. Encoder-only vs decoder-only?
Encoder-only reads bidirectionally; decoder-only predicts next tokens causally.
5. What is causal masking?
Preventing a token from attending to future tokens.
6. Why use residual connections?
They improve gradient flow and deep training.
7. Why LayerNorm instead of BatchNorm?
It works better for variable-length sequence features and small batches.
8. What is the main cost bottleneck?
Quadratic attention memory/time.
9. What is perplexity?
Exponentiated average negative log-likelihood for language models.
10. Why are Transformers powerful?
They combine parallel training with direct token-token interactions.

17. Practice Tasks

- Implement a tiny Transformer classifier.
- Add causal masking and train a character LM.
- Compare RNN and Transformer on text classification.
- Visualize attention heads.
- Fine-tune a Hugging Face model on a small dataset.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Resume Classifier	Categorizes resumes	Hugging Face, PyTorch	Resume dataset	Placement-relevant NLP
Mini GPT	Character-level generation	PyTorch	Shakespeare/text8	Shows fundamentals
FAQ Semantic Search	Embeds and retrieves FAQs	SentenceTransformers, FAISS	Company FAQs	AI engineering project

19. Quick Revision

- Key idea: self-attention plus feedforward blocks.
- Main formula: $\text{softmax}(QK^T/\sqrt{d_k})V$.
- When to use: language, vision, multimodal tasks.
- Important metrics: accuracy, F1, perplexity, BLEU/ROUGE.
- Common traps: masks and positional encoding.
- Interview one-liner: Transformers model contextual relationships through parallel self-attention.

20. Final Cheat Sheet

Item	Answer
Definition	Attention-based deep architecture
Input/output	Tokens/patches to contextual vectors/logits
Main steps	Embed, position, attention, FFN, output
Key hyperparameters	Layers, heads, d_model, context length
Metrics	Perplexity, F1, BLEU, ROUGE
Pros	Parallel, powerful, scalable
Cons	Expensive for long sequences
Best use cases	LLMs, NLP, ViT, multimodal AI

BERT

1. Overview

BERT (Bidirectional Encoder Representations from Transformers) is an encoder-only Transformer pretrained to build contextual token representations from both left and right context. The original model combines masked language modeling (MLM) with next sentence prediction (NSP), then fine-tunes a small task head with the encoder.

BERT is useful for text classification, named-entity recognition, extractive question answering, semantic matching, reranking, and domain-specific language understanding. It is primarily a representation model, not an autoregressive text generator.

2. Intuition

If the sentence is "The bank approved the loan," the meaning of "bank" depends on words on both sides. BERT hides some tokens and asks the model to recover them using the complete surrounding sentence. Repeating this over huge corpora teaches grammar, meaning, and relationships that transfer to labeled tasks.

3. Prerequisites

- Tokenization, embeddings, attention, multi-head attention, and positional encoding
- Encoder blocks, residual connections, LayerNorm, and feed-forward networks
- Softmax, cross-entropy, transfer learning, and fine-tuning
- PyTorch and Hugging Face `transformers`
- NLP evaluation: accuracy/F1, span exact match, and sequence labeling

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
Bidirectional self-attention	Every unmasked token attends left and right	Rich contextual understanding	"bank" uses "loan"	Why unsuitable for causal generation?
WordPiece	Subword tokenizer	Handles rare words with finite vocabulary	playing -> play ##ing	Token/label alignment
[CLS]	Special first token	Common sequence-level representation	Sentiment head	Is pooler always best?
[SEP]	Separates sequences	Supports sentence pairs	Premise/hypothesis	Segment IDs
MLM	Predict selected hidden tokens	Pretraining objective	Hide "approved"	Pretrain/fine-tune mismatch
NSP	Predict whether B follows A	Teaches sentence relation in original BERT	Pair classification	Why RoBERTa removed it

5. Algorithm / Working Process

1. Tokenize text into WordPiece IDs; add [CLS], [SEP], attention masks, and optional token-type IDs.
2. Add token, position, and segment embeddings.
3. Pass the sequence through stacked bidirectional Transformer encoders.
4. During pretraining, replace selected tokens using the 80/10/10 masking policy and predict their originals; optionally compute NSP.
5. For fine-tuning, attach a classifier, token classifier, or span head and update all or selected layers.
6. During inference, run one encoder pass and decode task-specific logits.

6. Mathematical Foundation

Self-attention in each head is

$$\text{Attention}(Q,K,V) = \text{softmax}(QK^T / \sqrt{d_k} + \text{mask})V$$

MLM minimizes negative log-likelihood only at masked positions M :

$$L_{\text{MLM}} = - \sum_{i \in M} \log p(x_i \mid x_{\text{without_masked_tokens}})$$

$$L_{\text{pretrain}} = L_{\text{MLM}} + L_{\text{NSP}}$$

For classification with [CLS] representation h_{cls} :

$$\text{logits} = W h_{\text{cls}} + b$$

$$L = - \sum_c y_c \log \text{softmax}(\text{logits})_c$$

Original BERT-Base uses 12 layers, hidden size 768, 12 heads, and about 110M parameters; BERT-Large uses 24 layers, size 1024, 16 heads, and about 340M parameters.

7. Practical Implementation

```
# pip install transformers torch
import torch
from transformers import AutoTokenizer, AutoModelForSequenceClassification

checkpoint = "bert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(checkpoint)
model = AutoModelForSequenceClassification.from_pretrained(
    checkpoint, num_labels=2
)

texts = ["The interview went very well.", "The service was disappointing."]
labels = torch.tensor([1, 0])
batch = tokenizer(texts, padding=True, truncation=True,
```

```

max_length=128, return_tensors="pt")

model.train()
optimizer = torch.optim.AdamW(model.parameters(), lr=2e-5)
outputs = model(**batch, labels=labels)
outputs.loss.backward()
torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
optimizer.step(); optimizer.zero_grad()

model.eval()
with torch.inference_mode():
    probabilities = model(**batch).logits.softmax(dim=-1)
print(probabilities)

```

8. Code Explanation

The tokenizer creates padded ID and attention-mask tensors. `AutoModelForSequenceClassification` loads the pretrained encoder and a randomly initialized classification head. Passing `labels` makes Hugging Face compute cross-entropy. Raw logits are used for training; softmax is applied only for interpretable inference probabilities. A small fine-tuning learning rate prevents destructive updates to pretrained features.

9. Training / Evaluation

Split by user, document, time, or source when random splitting would leak near-duplicates. Fit tokenization rules only through the fixed pretrained tokenizer. For classification use macro-F1 on imbalance; for NER use entity-level F1; for extractive QA use exact match and token F1. Tune learning rate ($1e-5$ – $5e-5$ is a common starting region), epochs, batch size, max length, weight decay, and warmup. Use early stopping, gradient clipping, mixed precision, and class weighting when justified.

10. Complexity and Cost

Full attention costs $O(L^2 d)$ time and $O(L^2)$ attention memory; feed-forward blocks cost roughly $O(L d^2)$. BERT encodes a sequence in parallel but cannot cache causal history like GPT. Fine-tuning BERT-Base is feasible on one GPU with modest sequence lengths; CPU inference is possible but latency-sensitive applications may need DistilBERT, quantization, or batching.

11. Common Use Cases

- Sentiment, intent, toxicity, and document classification
- Named-entity recognition and information extraction
- Extractive question answering
- Search reranking and sentence-pair classification
- Domain models such as BioBERT, ClinicalBERT, and LegalBERT

12. Common Mistakes

- Calling BERT a decoder or using it directly for left-to-right generation
- Applying softmax before cross-entropy
- Ignoring attention masks on padded batches

- Truncating away the evidence-bearing part of long documents
- Misaligning word-level NER labels with subword tokens
- Random splitting near-duplicate documents
- Using only accuracy for imbalanced tasks

13. Edge Cases / Limitations

BERT has a fixed context limit, quadratic attention, and costly pretraining. MLM pretraining never directly learns free-form generation. Static position limits and truncation hurt long documents. Biases in pretraining data transfer downstream. Confidence is often miscalibrated, and a plausible classification does not supply faithful reasoning.

14. Variations

Variation	Change	When to use	Importance
RoBERTa	More data/training, dynamic masking, no NSP	Stronger general encoder	High
DistilBERT	Knowledge-distilled smaller encoder	Low latency/memory	High projects
ALBERT	Parameter sharing/factorized embeddings	Parameter efficiency	Medium
DeBERTa	Disentangled content/position attention	Strong accuracy	Medium-high
Domain BERT	Continued domain pretraining	Specialized vocabulary/distribution	High projects

15. Related Topics

BERT versus GPT: BERT is bidirectional and encoder-only for understanding; GPT is causal and decoder-only for generation. BERT versus T5: T5 casts every task as text-to-text with encoder-decoder cross-attention. Sentence-BERT changes training/pooling to make independent sentence embeddings useful for cosine similarity. LayerNorm suits variable-length token batches better than BatchNorm.

16. Interview Questions

1. **What is BERT?** A bidirectional Transformer encoder pretrained mainly with masked-token prediction.
2. **Why is it bidirectional?** Non-causal attention allows each token to use left and right context.
3. **What is MLM?** Mask selected input tokens and predict their original IDs.
4. **Explain the 80/10/10 rule.** Of chosen prediction positions, 80% become [MASK], 10% random tokens, and 10% stay unchanged.
5. **What are token-type IDs?** Embeddings marking sentence A versus sentence B in paired inputs.
6. **What is [CLS] used for?** A sequence-level vector commonly fed to a classifier.
7. **Why did RoBERTa remove NSP?** Experiments found stronger training without the original NSP formulation.
8. **How is BERT adapted for NER?** Apply a token-classification head to each final token representation.
9. **Why is long text difficult?** Standard attention is quadratic and pretrained position length is limited.

10. **BERT versus Sentence-BERT?** Vanilla BERT cross-encoding is accurate but slow for retrieval; SBERT creates comparable independent embeddings.
11. **How do you prevent catastrophic forgetting?** Small learning rate, limited epochs, warmup, freezing, or parameter-efficient tuning.

17. Practice Tasks

- Fine-tune BERT for imbalanced sentiment and report macro-F1.
- Build a CoNLL-style NER pipeline with correct subword label alignment.
- Compare [CLS], mean pooling, and Sentence-BERT for semantic search.
- Debug a classifier whose validation set contains duplicate reviews.
- Add LoRA adapters and compare trainable parameters and accuracy.

18. Project Ideas

Project	What it does	Stack	Dataset	Resume value
Resume skill extractor	Finds skills, titles, and organizations	HF, PyTorch, FastAPI	Kaggle resumes + annotations	Placement-domain NLP
Support-ticket router	Predicts team and urgency	BERT, MLflow	CLINC150/custom tickets	Full evaluation/deployment
Search reranker	Reorders retrieved passages	Cross-encoder BERT	MS MARCO subset	Production retrieval skill

19. Quick Revision

- **Key idea:** bidirectional contextual encoder pretrained by masking tokens.
- **Main formula:** $L_{MLM} = -\sum \log p(\text{masked token} \mid \text{visible context})$.
- **Use:** classification, token labeling, extractive QA, reranking.
- **Metrics:** task-dependent F1, accuracy, EM, calibration, latency.
- **Traps:** leakage, truncation, missing masks, subword misalignment.
- **One-liner:** “BERT is an encoder-only Transformer that trades causal generation for deep bidirectional representations.”

20. Final Cheat Sheet

Item	Summary
Definition	Bidirectional pretrained Transformer encoder
Input/output	Token IDs/masks -> contextual token vectors or task logits
Steps	WordPiece -> embeddings -> encoder stack -> task head
Hyperparameters	LR, layers unfrozen, max length, batch size, warmup, weight decay
Metrics	Macro-F1/accuracy, entity F1, QA EM/F1, latency
Pros	Strong transfer learning; rich context; versatile heads

Item	Summary
Cons	Quadratic attention; fixed context; not naturally generative
Best use	Language-understanding and discriminative NLP tasks

GPT

1. Overview

GPT (Generative Pre-trained Transformer) is a family of decoder-only Transformers trained to predict the next token. Causal self-attention prevents a position from seeing future tokens, so the same objective used in pretraining directly supports open-ended generation. Modern GPT-style models are adapted through supervised fine-tuning, preference optimization, retrieval, tools, prompting, or parameter-efficient tuning.

GPT models power assistants, code completion, summarization, drafting, extraction, agents, and general-purpose language interfaces. “GPT” describes an architectural/training pattern; exact sizes and post-training recipes vary by model generation.

2. Intuition

Given “Deep learning models need lots of”, the model assigns probabilities to possible next tokens such as “data”. After choosing one token, it appends it and repeats. Learning this simple game across enormous and varied corpora forces the network to model syntax, facts, styles, code patterns, and some multi-step relationships.

3. Prerequisites

- Subword tokenization, embeddings, positional information, and softmax
- Transformer decoder blocks and causal self-attention
- Cross-entropy, maximum likelihood, gradient descent, and transfer learning
- Sampling: greedy, temperature, top-k, top-p, and stopping criteria
- Hugging Face/PyTorch inference and GPU memory basics

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
Causal mask	Position t cannot attend to $>t$	Prevents label leakage	Lower-triangular mask	GPT versus BERT
Next-token objective	Predict x_t from earlier tokens	One objective supports generation	Continue a sentence	Teacher forcing
Decoder-only block	Masked attention + MLP	Scalable autoregressive core	Repeated N times	No encoder cross-attention

Concept	Meaning	Why it matters	Example	Interview angle
Context window	Maximum usable tokens	Limits available evidence and cost	Prompt + answer tokens	Lost-in-the-middle issue
KV cache	Reuse past key/value tensors	Avoids recomputing full prefix	Token-by-token decoding	Memory grows with context
Post-training	Align pretrained behavior to instructions/preferences	Makes models useful and safer	SFT/DPO/RLHF	Pretraining vs alignment

5. Algorithm / Working Process

1. Tokenize a document into IDs and shift it into input/target pairs.
2. Add token and position representations.
3. Apply masked self-attention so hidden state h_t depends only on tokens $\leq t$.
4. Project each hidden state to V vocabulary logits and minimize next-token cross-entropy.
5. Optionally perform instruction tuning and preference/safety post-training.
6. At inference, tokenize the prompt, run a prefill pass, then decode one token at a time using cached keys/values.
7. Stop at EOS, a stop string, or a maximum-new-token budget.

6. Mathematical Foundation

Autoregressive factorization:

$$p(x_1, \dots, x_T) = \prod_{t=1}^T p(x_t \mid x_{<t})$$

$$L_{\text{NLL}} = -\sum_{t=1}^T \log p_{\theta}(x_t \mid x_{<t})$$

Causal attention uses a mask $M_{ij}=0$ for $j \leq i$ and $-\infty$ otherwise:

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k} + M)V$$

Temperature rescales logits before sampling:

$$p_i = \text{softmax}(\text{logit}_i / \tau)$$

Lower τ sharpens the distribution; higher τ increases diversity. Perplexity is $\exp(\text{average NLL})$ and measures average predictive uncertainty, not factuality or instruction quality.

7. Practical Implementation

```
# pip install transformers torch
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM

checkpoint = "distilgpt2"
tokenizer = AutoTokenizer.from_pretrained(checkpoint)
model = AutoModelForCausalLM.from_pretrained(checkpoint)
tokenizer.pad_token = tokenizer.eos_token

prompt = "In a machine-learning interview, explain overfitting:"
inputs = tokenizer(prompt, return_tensors="pt")

model.eval()
with torch.inference_mode():
    output_ids = model.generate(
        **inputs,
        max_new_tokens=80,
        do_sample=True,
        temperature=0.7,
        top_p=0.9,
        repetition_penalty=1.1,
        pad_token_id=tokenizer.eos_token_id,
    )

new_tokens = output_ids[0, inputs.input_ids.shape[1]:]
print(tokenizer.decode(new_tokens, skip_special_tokens=True))

# Fine-tuning batches use labels=input_ids; padding labels must be -100.
batch = tokenizer(["Deep learning uses neural networks."], return_tensors="pt")
labels = batch.input_ids.clone()
loss = model(**batch, labels=labels).loss
print(float(loss))
```

8. Code Explanation

`AutoModelForCausalLM` adds a vocabulary projection to a decoder-only backbone. `generate` performs autoregressive decoding; `top_p` restricts sampling to a probability mass and temperature controls sharpness. Slicing removes prompt tokens before decoding the completion. During training, the model internally shifts labels; `-100` positions are ignored by cross-entropy, which is essential for padding or prompt masking.

9. Training / Evaluation

Pretraining uses large deduplicated token corpora and document-aware splits. Fine-tuning formats examples consistently and often masks prompt tokens when only answer behavior should be learned. Evaluate language modeling with validation loss/perplexity; applications need task accuracy, pass@k for code, groundedness/factuality, toxicity/safety tests, calibration, human preference, latency, throughput, and cost. Tune learning rate, context length, batch/token budget, warmup, weight decay, decoding strategy, and stop conditions. Prevent benchmark contamination and test prompt-injection behavior for tool-using systems.

10. Complexity and Cost

Training full attention costs $O(L^2 d + Ld^2)$ per layer and stores large activations plus optimizer states. Autoregressive inference is sequential across generated tokens; KV caching removes repeated prefix computation but consumes approximately $O(\text{layers} * L * d)$ memory. Quantization lowers weight memory; batching improves throughput but can increase latency. Large GPT pretraining requires distributed GPU/accelerator clusters, whereas PEFT can adapt smaller open models on one or a few GPUs.

11. Common Use Cases

- Conversational assistants and natural-language interfaces
- Code completion, debugging, and test generation
- Drafting, summarization, extraction, and structured output
- Tool calling and agentic workflows
- Synthetic data and few-shot task adaptation

12. Common Mistakes

- Saying GPT uses bidirectional attention during ordinary generation
- Reporting perplexity as a factuality metric
- Decoding without EOS, stop, or token limits
- Using sampling for deterministic extraction without validation
- Fine-tuning on padded tokens or leaking answers into evaluation prompts
- Ignoring prompt-injection and unsafe tool permissions
- Assuming a larger context guarantees recall of every detail

13. Edge Cases / Limitations

GPT can hallucinate, inherit bias, expose memorized data, follow malicious instructions, and produce unstable answers across decoding settings. Causal generation is slow, context is finite, and self-attention is expensive. It does not inherently know current or private facts; retrieval can supply evidence but cannot guarantee faithful use. Arithmetic, exact counting, and long-horizon planning remain brittle without tools and checks.

14. Variations

Variation	Change	When to use	Importance
Instruction-tuned GPT	SFT on prompt-response data	Assistants and task following	Very high
Code model	Continued training on code	Software engineering	High placements
MoE decoder	Activates a subset of experts	Scale capacity efficiently	High research
Long-context model	RoPE scaling/sparse or optimized attention	Large documents	High projects
Multimodal GPT	Adds vision/audio encoders or tokens	Cross-modal assistants	High

15. Related Topics

GPT versus BERT: causal decoder generation versus bidirectional encoder understanding. GPT versus T5: decoder-only continuation versus encoder-decoder conditional generation. RNN language models carry a recurrent state; Transformers provide shorter gradient paths and parallel training. RAG changes the input context, while fine-tuning changes model parameters. LoRA/QLoRA reduce adaptation memory but do not provide fresh knowledge automatically.

16. Interview Questions

1. **What is GPT?** A decoder-only Transformer pretrained autoregressively.
2. **Why is a causal mask necessary?** It prevents a token from seeing the answer tokens to its right during training.
3. **What is teacher forcing?** Training predicts every next token using the true preceding sequence in parallel.
4. **Why is inference still sequential?** Token $t+1$ cannot be selected until token t is known.
5. **What does temperature do?** Rescales logits to adjust randomness.
6. **Top-k versus top-p?** Top-k keeps a fixed number of tokens; top-p keeps the smallest set reaching a probability mass.
7. **What is a KV cache?** Stored attention keys/values from previous positions reused during decoding.
8. **Why can KV cache become a bottleneck?** It grows with batch size, layers, context length, and hidden dimensions.
9. **What is perplexity?** Exponentiated average next-token NLL; lower means better prediction on that distribution.
10. **Pretraining versus fine-tuning?** Broad next-token learning versus narrower behavioral/task adaptation.
11. **How would you reduce hallucination?** Ground with retrieval/tools, constrain outputs, calibrate abstention, and verify claims; fine-tuning alone is insufficient.
12. **Why mask prompt loss in instruction tuning?** To focus gradient on desired response tokens.

17. Practice Tasks

- Implement a causal mask and verify future attention weights are zero.
- Fine-tune a small GPT on domain text and compare validation perplexity.
- Benchmark greedy, beam, top-k, and top-p decoding for quality/diversity.
- Diagnose repeated text and premature EOS in a generation pipeline.
- Add retrieval citations and measure whether answers are supported.

18. Project Ideas

Project	What it does	Stack	Dataset	Resume value
Interview coach	Generates and scores role-specific mock answers	HF, FastAPI, React	Curated Q&A + rubrics	LLM evaluation/product work
Code repair assistant	Proposes patches and runs tests in a sandbox	Transformers, Docker	BugsInPy/QuixBugs	Agent/tool safety

Project	What it does	Stack	Dataset	Resume value
Grounded policy bot	Answers with retrieved policy evidence	GPT-style LM, FAISS	Company handbook corpus	RAG and factuality metrics

19. Quick Revision

- **Key idea:** predict the next token with causal decoder blocks.
- **Main formula:** $p(x) = \text{product}_t p(x_t | x_{<t})$.
- **Use:** generation, assistants, code, and tool-driven workflows.
- **Metrics:** NLL/perplexity plus task, factuality, safety, latency, and cost.
- **Traps:** leakage, uncontrolled decoding, hallucination, prompt injection.
- **One-liner:** "GPT scales autoregressive next-token prediction into a general conditional generator."

20. Final Cheat Sheet

Item	Summary
Definition	Causal decoder-only pretrained Transformer
Input/output	Prompt tokens -> probability distribution and generated tokens
Steps	Tokenize -> causal blocks -> logits -> sample repeatedly
Hyperparameters	Context, LR, layers/width, temperature, top-p/k, max tokens
Metrics	Perplexity, task scores, groundedness, safety, latency/cost
Pros	General generation; in-context learning; scalable
Cons	Hallucination, sequential decoding, high compute/memory
Best use	Open-ended conditional text/code generation

T5

1. Overview

T5 (Text-to-Text Transfer Transformer) is an encoder-decoder Transformer that reformulates every NLP problem as text input mapped to text output. It is pretrained with a span-corruption denoising objective: contiguous token spans are replaced by unique sentinel tokens, and the decoder reconstructs the missing spans.

T5 supports translation, summarization, question answering, classification-as-generation, rewriting, and multitask learning through task prefixes such as `translate English to German:` or `summarize:`.

2. Intuition

Instead of building a different output head for every task, T5 makes the interface uniform. Input `â€œsentiment: this film was excellentâ€` produces output `â€œpositiveâ€`; a summarization prompt produces a summary. The encoder reads the entire request, and the decoder writes the answer while cross-attending to that encoded input.

3. Prerequisites

- Transformer encoder, causal decoder, and encoder-decoder cross-attention
- Subword tokenization and sequence-to-sequence teacher forcing
- Cross-entropy, masking, transfer learning, and beam/sampling decoding
- Hugging Face `transformers` and PyTorch
- NLP generation metrics and their limitations

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
Text-to-text	All targets are token sequences	One architecture/loss for many tasks	<code>sst2 sentence - > positive</code>	Cost of classification as generation
Encoder	Bidirectionally represents input	Strong source understanding	Reads full article	T5 versus GPT
Decoder	Causally generates target	Flexible output length	Writes summary	Teacher forcing
Cross-attention	Decoder queries encoder states	Conditions every output on source	Translation alignment	Q from decoder, K/V from encoder
Span corruption	Replace spans by sentinels	More efficient than independent masks	<code><extra_id_0></code>	T5 pretraining target
Task prefix	Natural-text task identifier	Enables multitask transfer	<code>summarize:</code>	Prompt consistency

5. Algorithm / Working Process

1. During pretraining, sample spans covering roughly a chosen fraction of tokens.
2. Replace each input span with a unique sentinel such as `<extra_id_0>`.
3. Construct the target by concatenating each sentinel and its removed span.
4. Encode the corrupted input with bidirectional self-attention.
5. Decode the target autoregressively with causal self-attention and cross-attention.
6. Fine-tune using task-prefixed input text and expected target text.
7. At inference, encode once and decode greedily, with beam search, or with sampling.

6. Mathematical Foundation

Conditional generation factorizes as

$$p(y \mid x) = \prod_{t=1}^T p(y_t \mid y_{<t}, \text{Encoder}(x))$$

$$L = -\sum_t \log p(y_t \mid y_{<t}, x)$$

Decoder cross-attention is

$$\text{CrossAttention}(Q_{\text{dec}}, K_{\text{enc}}, V_{\text{enc}})$$

$$= \text{softmax}(Q_{\text{dec}} K_{\text{enc}}^T / \sqrt{d_k}) V_{\text{enc}}$$

T5 uses relative position biases rather than the original Transformer's sinusoidal absolute positions. Padding labels are set to `-100` so they do not contribute to the loss. Sequence-level generation metrics should be interpreted alongside semantic/human evaluation.

7. Practical Implementation

```
# pip install transformers sentencepiece torch
import torch
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM

checkpoint = "google-t5/t5-small"
tokenizer = AutoTokenizer.from_pretrained(checkpoint)
model = AutoModelForSeq2SeqLM.from_pretrained(checkpoint)

source = ["summarize: Neural networks learn representations from data, "
          "but careful validation is required to measure generalization."]
target = ["Neural networks need careful validation."]

batch = tokenizer(source, padding=True, truncation=True,
                  max_length=256, return_tensors="pt")
labels = tokenizer(text_target=target, padding=True, truncation=True,
                  max_length=64, return_tensors="pt").input_ids
labels[labels == tokenizer.pad_token_id] = -100

model.train()
loss = model(**batch, labels=labels).loss
loss.backward()

model.eval()
with torch.inference_mode():
    ids = model.generate(**batch, max_new_tokens=48, num_beams=4,
                        length_penalty=1.0, early_stopping=True)
    print(tokenizer.batch_decode(ids, skip_special_tokens=True))
```

8. Code Explanation

The task prefix tells a multitask checkpoint which transformation to perform. Source and target have independent length limits. Replacing target padding with `-100` excludes it from cross-entropy. At inference

the encoder runs once; beam search retains several high-scoring partial outputs and can improve deterministic tasks, though it is slower and not automatically more factual.

9. Training / Evaluation

Split by document/source to avoid similar text crossing partitions. Inspect truncation rates separately for input and output. Use ROUGE for summarization, BLEU/COMET plus human review for translation, exact match/F1 for QA, and accuracy/macro-F1 after parsing classification labels. Validate malformed or out-of-label generations. Tune learning rate, source/target lengths, label smoothing, task mixture weights, beams, length penalty, and maximum output. For multitask training, prevent large datasets from overwhelming small tasks through temperature-scaled sampling or explicit weights.

10. Complexity and Cost

Encoder self-attention costs $O(S^2 d)$, decoder self-attention $O(T^2 d)$, and cross-attention $O(STd)$ per layer (plus feed-forward cost). Training stores both encoder and decoder activations. Inference encodes the source once but autoregressively decodes the target; beam search multiplies decoder work and cache memory. T5-small can run on CPU for demos; larger variants and long sequences benefit strongly from GPUs and mixed precision.

11. Common Use Cases

- Abstractive summarization and rewriting
- Translation and grammatical correction
- Generative question answering
- Multitask NLP through natural task prefixes
- Classification when a unified text interface is valuable

12. Common Mistakes

- Omitting or inconsistently wording task prefixes
- Computing loss on target padding tokens
- Using one max length for source and target without inspection
- Treating ROUGE/BLEU as complete semantic/factual evaluation
- Expecting beam search to eliminate hallucination
- Failing to constrain labels for classification-as-generation
- Confusing decoder self-attention with encoder-decoder cross-attention

13. Edge Cases / Limitations

Generating label strings is slower and less constrained than a classification head. T5 can hallucinate in summarization and QA, especially when inputs lack evidence. Long sources face quadratic encoder cost and truncation. Multi-task mixtures can cause negative transfer. Small wording changes to prefixes may alter behavior, and automatic overlap metrics penalize valid paraphrases or reward unsupported copying.

14. Variations

Variation	Change	When to use	Importance
FLAN-T5	Instruction tuning across many tasks	Zero/few-shot task following	Very high
mT5	Multilingual pretraining	Cross-lingual tasks	High
ByT5	Byte-level inputs	Noisy text and broad scripts	Medium
UL2	Mixture of denoising objectives	More general conditional generation	Research
LongT5	Long-input attention mechanisms	Documents and summarization	High projects

15. Related Topics

T5 versus BERT: both use bidirectional encoders, but T5 adds an autoregressive decoder and generates every output. T5 versus GPT: T5 explicitly separates source encoding from target decoding; GPT concatenates context and continuation in one causal stream. BART is also a denoising encoder-decoder but uses different corruption and architecture choices. RAG can use T5 as its answer generator after retrieval.

16. Interview Questions

1. **What does text-to-text mean?** Inputs and outputs for every task are represented as token sequences.
2. **What is T5's pretraining objective?** Reconstruct removed contiguous spans identified by sentinel tokens.
3. **Why sentinel tokens?** They mark distinct missing spans in both corrupted input and reconstruction target.
4. **Where is attention bidirectional?** In the encoder; decoder self-attention remains causal.
5. **What is cross-attention?** Decoder queries attend to encoder keys and values.
6. **T5 versus GPT?** Encoder-decoder conditional generation versus a decoder-only continuation model.
7. **Why use task prefixes?** They disambiguate the desired mapping in a shared multitask interface.
8. **Why set pad labels to -100?** PyTorch cross-entropy ignores those positions.
9. **When is beam search useful?** Structured, relatively deterministic sequence generation such as translation; it trades compute for search breadth.
10. **Why can ROUGE be misleading?** Lexical overlap neither guarantees factuality nor recognizes every valid paraphrase.
11. **How would you do classification?** Generate a canonical label and validate/constrain it, or attach a discriminative head if speed matters.

17. Practice Tasks

- Fine-tune T5-small for headline generation and measure ROUGE plus factual errors.
- Convert a three-class classifier into text-to-text format and validate labels.
- Implement span corruption with sentinel-token targets.
- Diagnose repetitive or empty summaries by inspecting labels and decoding settings.
- Multi-task two datasets and compare uniform versus size-proportional sampling.

18. Project Ideas

Project	What it does	Stack	Dataset	Resume value
Meeting action extractor	Converts transcripts into structured action items	FLAN-T5, FastAPI	AMI Meeting Corpus	Seq2seq + structured validation
Multilingual FAQ reformulator	Rewrites questions into search-friendly form	mT5, Elasticsearch	MIRACL/custom FAQs	Multilingual retrieval impact
Explainable classifier	Outputs label plus concise rationale	T5, PyTorch, MLflow	e-SNLI	Multitask/evaluation depth

19. Quick Revision

- **Key idea:** one encoder-decoder, every task expressed as text-to-text.
- **Main formula:** $p(y|x) = \text{product}_t p(y_t | y_{<t}, \text{Encoder}(x))$.
- **Use:** summarization, translation, rewriting, QA, multitask NLP.
- **Metrics:** ROUGE, BLEU/COMET, EM/F1, label validity, factuality, latency.
- **Traps:** pad loss, truncation, prefix mismatch, metric-only evaluation.
- **One-liner:** "T5 unifies NLP tasks as conditional generation and pretrains by reconstructing corrupted text spans."

20. Final Cheat Sheet

Item	Summary
Definition	Text-to-text encoder-decoder Transformer
Input/output	Task-prefixed source text -> generated target text
Steps	Encode source -> causal decode with cross-attention -> detokenize
Hyperparameters	LR, input/output lengths, task weights, beams, length penalty
Metrics	ROUGE/BLEU/COMET, EM/F1, factuality, label validity
Pros	Unified interface; strong conditional generation; multitask transfer
Cons	Decoder latency; unconstrained outputs; two-stack memory cost
Best use	Source-conditioned generation and multitask NLP

ResNet

1. Overview

ResNet, or Residual Network, is a deep CNN architecture that uses skip connections to train very deep networks effectively. It was a major breakthrough in computer vision.

ResNets are used as backbones for classification, detection, segmentation, medical imaging, satellite vision, and feature extraction.

2. Intuition

Instead of forcing each block to learn a complete transformation, ResNet lets it learn a correction to the input. If a layer is not needed, it can learn near-zero residual and pass information through.

3. Prerequisites

- CNNs
- Backpropagation
- Vanishing gradients
- Batch normalization
- Residual connections

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Skip connection	Adds input to block output	Improves gradient flow	$y = F(x) + x$	Why ResNet trains deep
Residual function	Learns correction	Easier optimization	edge refinement	Residual learning
Basic block	Two 3x3 convs	ResNet-18/34	simple residual block	Architecture difference
Bottleneck block	1x1, 3x3, 1x1 convs	Efficient deep ResNets	ResNet-50	Why bottleneck
Projection shortcut	Matches dimensions	Needed when channels/size change	1x1 conv skip	Shape compatibility

5. Algorithm / Working Process

Input image passes through initial conv and pooling. Residual stages apply blocks. Each block computes $F(x)$ and adds shortcut x . Spatial size usually decreases stage by stage while channels increase. Global average pooling and linear layer produce logits.

6. Mathematical Foundation

Residual block:

$$y = F(x, W) + x$$

If dimensions differ:

$$y = F(x, W) + W_s x$$

Gradient benefit:

$$dL/dx = dL/dy * (dF/dx + I)$$

The identity term helps gradients flow backward.

7. Practical Implementation

```
import torch
from torch import nn

class BasicBlock(nn.Module):
    def __init__(self, channels):
        super().__init__()
        self.block = nn.Sequential(
            nn.Conv2d(channels, channels, 3, padding=1, bias=False),
            nn.BatchNorm2d(channels),
            nn.ReLU(),
            nn.Conv2d(channels, channels, 3, padding=1, bias=False),
            nn.BatchNorm2d(channels),
        )
        self.relu = nn.ReLU()

    def forward(self, x):
        return self.relu(self.block(x) + x)

x = torch.randn(4, 64, 32, 32)
block = BasicBlock(64)
print(block(x).shape)
```

8. Code Explanation

The block applies two convolution layers and adds the original input before final ReLU. Because channels and spatial size stay the same, no projection shortcut is needed.

9. Training / Evaluation

Use standard image augmentation, normalization, SGD/AdamW, weight decay, and learning rate schedules. Metrics include top-1/top-5 accuracy, F1, mAP for detection, and IoU for segmentation when ResNet is used as backbone.

10. Complexity and Cost

Cost depends on depth: ResNet-18 is lightweight, ResNet-50 is common, ResNet-101/152 are heavier. Residual connections add little compute but greatly improve optimization.

11. Common Use Cases

- Image classification
- Object detection backbone
- Segmentation encoder
- Medical image feature extractor
- Transfer learning

12. Common Mistakes

- Adding tensors with mismatched shapes
- Forgetting projection shortcut when channels change
- Training very deep CNN without normalization
- Using ImageNet normalization incorrectly
- Fine-tuning all layers on tiny data without regularization

13. Edge Cases / Limitations

ResNets still have limited global context compared with attention models. Large ResNets can be costly for mobile deployment.

14. Variations

Variation	What changes	When to use	Importance
ResNet-18/34	Basic blocks	Faster tasks	High
ResNet-50/101	Bottleneck blocks	Stronger accuracy	High
ResNeXt	Grouped convolutions	Better accuracy/efficiency	Medium
Wide ResNet	Wider channels	CIFAR-style tasks	Medium

15. Related Topics

ResNet is a CNN architecture. U-Net often uses residual encoders. Transformers also use residual connections. DenseNet extends feature reuse by concatenation instead of addition.

16. Interview Questions

1. What problem did ResNet solve?
Training degradation in very deep networks.
2. What is a residual connection?
Adding block input to block output.
3. Why does $F(x)+x$ help?
It improves gradient flow and lets blocks learn corrections.

4. When need projection shortcut?
When channel count or spatial size changes.
5. Basic block vs bottleneck?
Basic uses 3x3 convs; bottleneck uses 1x1-3x3-1x1.
6. Why use global average pooling?
It reduces parameters compared with flattening.
7. ResNet vs plain CNN?
ResNet trains much deeper models.
8. What is degradation problem?
Deeper plain networks perform worse despite more capacity.
9. Is skip connection only for CNNs?
No, Transformers also use residual connections.
10. Why is ResNet popular for transfer learning?
Strong pretrained visual features.

17. Practice Tasks

- Implement a residual block.
- Train ResNet-18 with transfer learning.
- Compare plain CNN vs ResNet on CIFAR-10.
- Add projection shortcut for downsampling.
- Visualize feature maps.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Medical X-ray Classifier	Detects disease	PyTorch, torchvision	ChestX-ray14	Healthcare CV
Product Image Classifier	Categorizes products	PyTorch	E-commerce images	Industry CV
Transfer Learning Benchmark	Compares ResNet variants	PyTorch	CIFAR/Food-101	Strong experimentation

19. Quick Revision

- Key idea: learn residual corrections with skip connections.
- Main formula: $y = F(x) + x$.
- When to use: deep CNN backbones.
- Important metrics: accuracy, mAP, IoU.
- Common traps: shortcut shape mismatch.
- Interview one-liner: ResNet made very deep CNNs trainable using identity shortcuts.

20. Final Cheat Sheet

Item	Answer
Definition	CNN with residual skip connections
Input/output	Image to features/logits
Main steps	Conv block plus shortcut addition
Key hyperparameters	Depth, block type, channels
Metrics	Accuracy, mAP, IoU
Pros	Trains deep models well
Cons	Less global context than attention
Best use cases	Vision backbones and transfer learning

DenseNet

1. Overview

DenseNet (Densely Connected Convolutional Network) connects every layer inside a dense block to every later layer by feature-map concatenation. Instead of repeatedly replacing features, layer 1 receives all earlier block outputs and contributes a small set of new maps. This encourages feature reuse, strengthens gradient flow, and can achieve high accuracy with fewer parameters than similarly deep conventional CNNs.

DenseNet is used as an image-classification backbone, a medical-imaging feature extractor, and an encoder in detection or segmentation systems, especially where data efficiency matters.

2. Intuition

Imagine every team member can read every earlier member's notes before adding a short new observation. No one must rediscover an edge or texture that an earlier layer already found. Concatenation preserves those old notes exactly, while each new layer only needs to add k useful feature maps.

3. Prerequisites

- Convolution, feature maps, channels, receptive fields, and pooling
- BatchNorm, ReLU, backpropagation, and vanishing gradients
- Residual/skip connections and concatenation versus addition
- Global average pooling and image-classification metrics
- PyTorch/torchvision transfer learning

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
Dense connectivity	Each layer sees all prior block outputs	Feature reuse and short gradient paths	$x_1 = H_1([x_0 \dots x_{l-1}])$	Compare with ResNet addition

Concept	Meaning	Why it matters	Example	Interview angle
Growth rate k	New channels produced per layer	Controls width/compute	$k=32$	Why can DenseNet be narrow?
Dense block	Stack at fixed resolution	Accumulates features	6/12/24/16 layers	Channel growth formula
Bottleneck	1×1 conv before 3×3	Reduces expensive input channels	DenseNet-BC	Parameter efficiency
Transition layer	1×1 conv plus average pool	Compresses and downsamples	Between dense blocks	Compression factor
Concatenation	Channel-wise join	Keeps feature identity	$[old, new]$	Memory cost

5. Algorithm / Working Process

1. Apply a stem convolution/pooling to the image.
2. In a dense block, compute normalized/transformed new features from the concatenation of all current features.
3. Append those k features to the block state.
4. Use a transition layer to compress channels and halve spatial resolution.
5. Repeat dense blocks/transitions; apply final normalization and global average pooling.
6. Train the classifier with cross-entropy and backpropagation through many short connectivity paths.
7. For transfer inference, preprocess with the checkpoint's normalization and replace/use the final head.

6. Mathematical Foundation

For dense layer l :

$$x_l = H_l([x_0, x_1, \dots, x_{l-1}])$$

$$C_l = C_0 + l * k$$

$[.]$ is channel concatenation and H_l is typically BN-ReLU- 1×1 Conv-BN-ReLU- 3×3 Conv. With compression θ in a transition:

$$C_{out} = \text{floor}(\theta * C_{in}), \quad 0 < \theta \leq 1$$

Classification uses $p = \text{softmax}(W \cdot \text{GAP}(x) + b)$ and cross-entropy $L = -\log p_y$. A dense block has $O(L^2)$ connections, though not necessarily $O(L^2)$ parameters because each layer only emits k maps.

7. Practical Implementation

```

import torch
from torch import nn
from torchvision.models import densenet121, DenseNet121_Weights

weights = DenseNet121_Weights.DEFAULT
model = densenet121(weights=weights)

# Replace ImageNet's 1000-class classifier for a 5-class problem.
model.classifier = nn.Linear(model.classifier.in_features, 5)
preprocess = weights.transforms()

# A batch would normally be built by a Dataset applying `preprocess`.
images = torch.randn(4, 3, 224, 224)
labels = torch.tensor([0, 1, 2, 3])
logits = model(images)
loss = nn.CrossEntropyLoss()(logits, labels)
loss.backward()

assert logits.shape == (4, 5)
print(sum(p.numel() for p in model.parameters()))

```

8. Code Explanation

`DenseNet121_Weights.DEFAULT` loads pretrained weights and exposes the exact resize/normalization pipeline. The only architectural edit is replacing `model.classifier`; its input size is discovered from the checkpoint. The model returns raw logits for `CrossEntropyLoss`. In real training, imagesâ€”not already normalized random tensorsâ€”must pass through `preprocess` or an equivalent augmentation pipeline.

9. Training / Evaluation

Use stratified or group-aware splits and fit augmentation only to training data. Start with a frozen backbone and train the head, then unfreeze later blocks with a smaller learning rate. Track top-1/top-k accuracy or macro-F1 for imbalance, plus calibration and per-class recall. Tune learning rate, growth architecture/checkpoint, weight decay, augmentation, dropout, resolution, and batch size. Dense connections improve gradients but do not eliminate overfitting; early stopping and transfer learning remain important.

10. Complexity and Cost

Each `3x3` convolution costs roughly $O(HW * C_{in} * k * 9)$; `Cin` grows through a block. DenseNet can use fewer parameters through small growth rates, but training activation memory and concatenation traffic are substantial because early maps remain live. Memory-efficient checkpointing recomputes activations to reduce storage. GPU training is preferred; CPU inference is practical at small batch sizes but may be slower than similarly accurate modern efficient backbones.

11. Common Use Cases

- Medical X-ray and pathology classification
- General transfer-learning image classifiers
- Feature extractors for detection and segmentation

- Small/medium datasets benefiting from feature reuse
- Research on connectivity and gradient propagation

12. Common Mistakes

- Saying DenseNet adds features like ResNet; it concatenates them
- Confusing depth with the number of dense blocks
- Ignoring checkpoint-specific normalization
- Reinitializing pretrained features accidentally
- Assuming fewer parameters means lower activation memory or latency
- Randomly splitting multiple images from the same patient/entity
- Applying softmax before cross-entropy

13. Edge Cases / Limitations

Concatenation causes high activation memory, memory-bandwidth pressure, and implementation overhead. Very large images or dense prediction can be expensive. DenseNet may be slower than its parameter count suggests, and modern ConvNeXt/EfficientNet/ViT backbones may offer better deployment trade-offs. Global feature reuse does not by itself solve distribution shift or poor labels.

14. Variations

Variation	Change	When to use	Importance
DenseNet-121/169/201	Different block depths	Accuracy/compute trade-off	High
DenseNet-BC	Bottlenecks plus compressed transitions	Standard efficient form	Very high
FC-DenseNet/Tiramisu	Dense blocks for segmentation decoder	Pixel prediction	Medium
CondenseNet	Learned group convolution/pruning	Mobile efficiency	Research

15. Related Topics

DenseNet concatenates all prior block features; ResNet adds a block output to one identity path. U-Net concatenates encoder features at corresponding decoder resolutions, not every prior layer. CSPNet splits feature flow to reduce DenseNet-style duplication. EfficientNet focuses on balanced depth/width/resolution scaling rather than dense connectivity.

16. Interview Questions

1. **What is DenseNet's defining equation?** $x_l = H_l([x_0, \dots, x_{l-1}])$.
2. **DenseNet versus ResNet skip connections?** Concatenation versus element-wise addition.
3. **What is growth rate?** The number of new feature maps each dense layer adds.
4. **Why can DenseNet use fewer parameters?** Later layers reuse earlier maps instead of relearning/recreating them.
5. **What is a transition layer?** Channel compression followed by spatial downsampling.

6. **What does DenseNet-BC mean?** Bottleneck layers plus compression.
7. **Why is gradient flow strong?** Loss gradients have short paths to early layers.
8. **What is the main systems drawback?** Retaining/concatenating many feature maps costs activation memory and bandwidth.
9. **Does fewer parameters guarantee faster inference?** No; concatenation and memory movement can dominate.
10. **When prefer DenseNet?** Strong transfer features and parameter efficiency matter more than minimal activation memory.
11. **How do channels grow?** Approximately $C_0 + L \cdot k$ within a block before compression.

17. Practice Tasks

- Implement a three-layer dense block and assert channel counts.
- Fine-tune DenseNet-121 on a five-class dataset with group-aware splits.
- Compare ResNet-50 and DenseNet-121 parameters, latency, memory, and F1.
- Debug an out-of-memory run using checkpointing and smaller resolution.
- Visualize Grad-CAM and audit spurious image regions.

18. Project Ideas

Project	What it does	Stack	Dataset	Resume value
Chest finding classifier	Multi-label X-ray findings with calibration	PyTorch, torchvision	ChestX-ray14	Medical CV/evaluation
Plant disease app	Mobile-facing leaf diagnosis API	DenseNet, FastAPI	PlantVillage	Transfer/deployment
Backbone benchmark	Compares DenseNet/ResNet/EfficientNet	PyTorch, MLflow	Food-101 subset	Systems + experiment rigor

19. Quick Revision

- **Key idea:** every layer reuses all earlier block features through concatenation.
- **Main formula:** $x_l = H_l([x_0, \dots, x_{l-1}])$, channels grow by k .
- **Use:** parameter-efficient classification and transfer learning.
- **Metrics:** task score plus parameter count, memory, and latency.
- **Traps:** confusing concat with addition; underestimating activation cost.
- **One-liner:** "DenseNet pays activation memory to buy feature reuse and exceptionally short gradient paths."

20. Final Cheat Sheet

Item	Summary
Definition	CNN with all-to-later concatenations inside dense blocks
Input/output	Image -> class logits/features
Steps	Stem -> dense blocks -> transitions -> GAP -> classifier

Item	Summary
Hyperparameters	Growth rate, block depths, compression, LR, resolution
Metrics	Accuracy/F1, calibration, parameters, memory, latency
Pros	Feature reuse, strong gradients, parameter efficiency
Cons	Activation memory, concatenation overhead, bandwidth cost
Best use	Transfer learning where rich reusable features matter

EfficientNet

1. Overview

EfficientNet is a CNN family designed to improve accuracy per parameter and FLOP by jointly scaling network depth, width, and input resolution. It starts from an efficiently searched baseline, EfficientNet-B0, and applies compound scaling to produce B1–B7. Its building blocks use mobile inverted bottleneck convolutions (MBConv), depthwise separable convolution, squeeze-and-excitation (SE), and skip connections.

EfficientNet is used for image classification and as a compact backbone in detection/segmentation, especially when latency, memory, or model size must be balanced against accuracy.

2. Intuition

Making only a network deeper may miss fine image detail; only widening it is expensive; only increasing resolution gives the same weak network more pixels. EfficientNet turns one “model size” dial that increases all three in a balanced, empirically chosen ratio—like upgrading CPU, memory, and display together rather than creating one bottleneck.

3. Prerequisites

- CNNs, receptive fields, stride, BatchNorm, activations, and residual paths
- Depthwise separable and 1×1 pointwise convolution
- Inverted bottlenecks and squeeze-and-excitation attention
- FLOPs, parameter count, latency, and transfer learning
- PyTorch/torchvision and image preprocessing

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
Compound scaling	Scale depth/width/resolution together	Better balanced accuracy-cost curve	B0 -> B7	Scaling equation/constraint
MBConv	Expand, depthwise conv, SE, project	Efficient spatial/channel mixing	Expansion ratio 6	Inverted residual

Concept	Meaning	Why it matters	Example	Interview angle
Depthwise conv	One spatial kernel per channel	Far cheaper than full conv	<code>groups=Cin</code>	Cost comparison
SE block	Learns channel gates from global context	Reweights informative channels	Sigmoid channel weights	Channel attention
Swish/SiLU	Smooth $x * \text{sigmoid}(x)$ activation	Often improves accuracy	<code>nn.SiLU()</code>	ReLU trade-off
Stochastic depth	Randomly drops residual branches in training	Regularizes deep variants	Survival probability	Train/eval behavior

5. Algorithm / Working Process

1. Resize/crop and normalize the input for the chosen checkpoint.
2. Apply a convolutional stem.
3. In each MBConv, optionally expand channels with `1x1` convolution.
4. Apply depthwise spatial convolution, SE channel gating, and linear `1x1` projection.
5. Add a residual connection when stride and channel dimensions match; optionally use stochastic depth.
6. Repeat stage configurations, then global-average-pool and classify.
7. Scale B0 with a compound coefficient or select a pretrained B-variant for the target budget.

6. Mathematical Foundation

Compound scaling uses coefficient `phi`:

```
depth = alpha^phi
width = beta^phi
resolution = gamma^phi
subject approximately to alpha * beta^2 * gamma^2
```

The exponents reflect convolutional cost: roughly linear in depth and quadratic in width/resolution. Standard versus depthwise-separable convolution cost:

```
standard: H*W*Cin*Cout*k^2
separable: H*W*(Cin*k^2 + Cin*Cout)
```

SE computes $s = \text{sigmoid}(W_2 \text{ activation}(W_1 \text{ GAP}(x)))$ and returns $s \odot x$. SiLU is $x * \text{sigmoid}(x)$. Training commonly uses cross-entropy, often with label smoothing.

7. Practical Implementation

```

import torch
from torch import nn
from torchvision.models import efficientnet_b0, EfficientNet_B0_Weights

weights = EfficientNet_B0_Weights.DEFAULT
model = efficientnet_b0(weights=weights)
preprocess = weights.transforms()

# torchvision's classifier is Dropout followed by Linear.
in_features = model.classifier[1].in_features
model.classifier[1] = nn.Linear(in_features, 10)

images = torch.randn(8, 3, 224, 224)
labels = torch.randint(0, 10, (8,))
optimizer = torch.optim.AdamW(model.parameters(), lr=3e-4)

model.train(); optimizer.zero_grad()
logits = model(images)
loss = nn.CrossEntropyLoss(label_smoothing=0.1)(logits, labels)
loss.backward(); optimizer.step()

model.eval()
with torch.inference_mode():
    predictions = model(images).argmax(dim=1)
assert predictions.shape == (8,)

```

8. Code Explanation

The pretrained weight enum supplies both parameters and the expected transforms. Replacing only the last linear layer preserves the feature extractor. `AdamW` and modest label smoothing are reasonable fine-tuning choices, but validation should decide. `inference_mode` disables autograd overhead; deployment measurements must include the actual preprocessing and hardware.

9. Training / Evaluation

Use stratified/group-aware train-validation-test splits and strong but label-preserving augmentation. Match pretrained resize, crop, and normalization. Evaluate accuracy/top-k or macro-F1 plus calibration; for deployment also measure end-to-end p50/p95 latency, throughput, peak memory, energy, and model size on target hardware. Tune variant, input resolution, augmentation, dropout/stochastic depth, learning rate, weight decay, and fine-tuning schedule. Larger nominal B variants are not automatically best when data or latency is limited.

10. Complexity and Cost

Depthwise separable convolutions sharply reduce multiply-adds and parameters, but realized speed depends on kernel support and memory behavior. Increasing resolution raises activation memory and convolution cost approximately quadratically. B0/B1 are friendly to a single GPU and edge-oriented deployment; large variants require more memory. FLOPs are a hardware-independent proxy, not latency—always benchmark the exported model on the device.

11. Common Use Cases

- Mobile/edge image classification
- Cloud services needing strong throughput per GPU
- Detection and segmentation backbones
- Transfer learning on modest datasets
- Medical, agricultural, retail, and manufacturing vision

12. Common Mistakes

- Scaling only image resolution and calling it compound scaling
- Assuming fewer FLOPs always means lower latency
- Using incorrect pretrained normalization or resolution
- Replacing the wrong classifier submodule
- Comparing models at different preprocessing/precision/batch sizes
- Choosing a huge variant for a small dataset
- Ignoring class imbalance and entity leakage

13. Edge Cases / Limitations

Depthwise kernels may be poorly optimized on some CPUs/accelerators. High-resolution variants can exceed memory despite modest parameter counts. Architecture search/scaling found on one resource regime may not be optimal for another. EfficientNet still has a CNN's locality bias and can struggle with large distribution shifts, tiny objects after resizing, and tasks needing global context.

14. Variations

Variation	Change	When to use	Importance
B0-B7	Compound-scaled depth/width/resolution	Choose capacity budget	Very high
EfficientNetV2	Fused MBConv and training-aware scaling	Faster training/inference	High
EfficientNet-Lite	Mobile-friendly operators	TFLite/edge deployment	High projects
EfficientDet	BiFPN plus compound detector scaling	Object detection	High CV roles

15. Related Topics

MobileNet introduced depthwise separable/mobile bottleneck ideas central to MBConv. ResNet emphasizes residual depth and simple blocks; DenseNet emphasizes concatenative reuse. EfficientNet adds SE and compound scaling to an efficient mobile-style backbone. ConvNeXt modernizes a conventional CNN, while Vision Transformers trade convolutional inductive bias for token attention.

16. Interview Questions

1. **What problem does EfficientNet solve?** Balanced accuracy versus compute/model size through coordinated scaling.
2. **What is compound scaling?** Increasing depth, width, and resolution together using fixed ratios and a coefficient.

3. **Why is width squared in the compute constraint?** Convolution connects input and output channels, so scaling both multiplies cost roughly quadratically.
4. **What is MBConv?** An inverted bottleneck with expansion, depthwise convolution, SE, and projection.
5. **Why is an inverted bottleneck?** The middle representation is wider; the block begins/ends narrow.
6. **Why use depthwise convolution?** It performs spatial filtering per channel much more cheaply than full convolution.
7. **What does SE do?** Global context produces channel-wise multiplicative gates.
8. **Why is linear projection used?** A nonlinear activation in the narrow bottleneck can destroy information.
9. **EfficientNet-B0 versus B7?** Same family, but B7 is deeper, wider, and trained at higher resolution.
10. **Why can low FLOPs still be slow?** Hardware kernels, memory access, launch overhead, and batch shape matter.
11. **How would you select a variant?** Benchmark validation quality and full pipeline latency/memory on target hardware.

17. Practice Tasks

- Derive standard versus depthwise-separable convolution FLOPs.
- Fine-tune B0 and B2 at their expected preprocessing sizes.
- Benchmark CPU/GPU/ONNX latency at batch 1 and batch 32.
- Diagnose accuracy loss caused by incorrect normalization or aggressive resizing.
- Replace an MBConv SE block and measure the quality-cost change.

18. Project Ideas

Project	What it does	Stack	Dataset	Resume value
Edge waste sorter	Classifies waste on a small device	PyTorch, ONNX/TFLite	TrashNet	Deployment benchmarking
Surface defect inspector	Detects manufacturing defects	EfficientNet, OpenCV	NEU Surface Defect	Industrial CV
Model efficiency dashboard	Compares backbones across devices	PyTorch, ONNX, Streamlit	CIFAR-100/ImageNet subset	MLOps/systems evidence

19. Quick Revision

- **Key idea:** balanced depth-width-resolution scaling around MBConv blocks.
- **Main formula:** $d=\alpha^\phi$, $w=\beta^\phi$, $r=\gamma^\phi$ with compute constraint.
- **Use:** high-accuracy classification under compute/model budgets.
- **Metrics:** task quality plus real latency, memory, size, throughput.
- **Traps:** equating FLOPs with speed; preprocessing mismatch; oversized variant.
- **One-liner:** "EfficientNet couples an efficient mobile-style block with compound scaling so capacity and resolution grow together."

20. Final Cheat Sheet

Item	Summary
Definition	Compound-scaled CNN family built from MBConv/SE blocks
Input/output	Image -> logits/features
Steps	Stem -> MBConv stages -> pooling -> classifier
Hyperparameters	Variant/phi, resolution, LR, dropout, stochastic depth
Metrics	Accuracy/F1, FLOPs, device latency, memory, model size
Pros	Excellent accuracy-efficiency trade-off; good transfer models
Cons	Operator-dependent speed; resolution memory; tuning complexity
Best use	Resource-aware vision classification/backbones

U-Net

1. Overview

U-Net is an encoder-decoder CNN architecture for dense prediction, especially image segmentation. It has a contracting path for context and an expanding path for precise localization, connected by skip connections.

It is widely used in biomedical segmentation, satellite imagery, road extraction, industrial inspection, and image-to-image tasks.

2. Intuition

The encoder asks "what is in the image?", while the decoder asks "where exactly is it?" Skip connections pass high-resolution details from encoder to decoder.

3. Prerequisites

- CNNs
- Encoder-decoder architecture
- Upsampling/transposed convolution
- Segmentation masks
- Dice/IoU metrics

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Contracting path	Downsampling encoder	Captures context	Organ shape	Why downsample
Expanding path	Upsampling decoder	Restores resolution	Pixel mask	Transposed conv
Skip connections	Copy encoder features	Preserves boundaries	Edge details	U-Net vs autoencoder

Subtopic	Meaning	Why it matters	Example	Interview angle
Segmentation mask	Per-pixel label	Dense prediction output	tumor mask	Loss and metrics
Dice loss	Overlap-based loss	Handles imbalance	small lesion	Dice vs IoU

5. Algorithm / Working Process

Input image goes through encoder blocks with pooling. Decoder upsamples features step by step. At each decoder stage, it concatenates corresponding encoder features. Final 1x1 convolution predicts class logits for every pixel.

6. Mathematical Foundation

Binary Dice coefficient:

$$\text{Dice} = \frac{2|P \text{ intersect } Y|}{(|P| + |Y|)}$$

$$\text{DiceLoss} = 1 - \text{Dice}$$

Pixel-wise cross-entropy:

$$L = -\sum_{\text{pixels}} \sum_{\text{classes}} y_c \log(p_c)$$

7. Practical Implementation

```
import torch
from torch import nn

def conv_block(cin, cout):
    return nn.Sequential(
        nn.Conv2d(cin, cout, 3, padding=1),
        nn.ReLU(),
        nn.Conv2d(cout, cout, 3, padding=1),
        nn.ReLU(),
    )

class MiniUNet(nn.Module):
    def __init__(self, classes=1):
        super().__init__()
        self.enc1 = conv_block(3, 32)
        self.pool = nn.MaxPool2d(2)
        self.enc2 = conv_block(32, 64)
        self.up = nn.ConvTranspose2d(64, 32, 2, stride=2)
        self.dec1 = conv_block(64, 32)
        self.out = nn.Conv2d(32, classes, 1)
```

```
def forward(self, x):
    e1 = self.enc1(x)
    e2 = self.enc2(self.pool(e1))
    d1 = self.up(e2)
    d1 = torch.cat([d1, e1], dim=1)
    return self.out(self.dec1(d1))

model = MiniUNet()
print(model(torch.randn(2, 3, 128, 128)).shape)
```

8. Code Explanation

`enc1` keeps high-resolution features. `pool` downsamples. `ConvTranspose2d` upsamples. `torch.cat` joins decoder and encoder features. The final `1x1` convolution predicts per-pixel logits.

9. Training / Evaluation

Prepare image-mask pairs with identical geometric augmentations. Use BCE/Dice for binary segmentation or cross-entropy/Dice for multi-class. Metrics include IoU, Dice, pixel accuracy, precision, and recall. Use patch training for large images.

10. Complexity and Cost

Memory is high because skip features must be stored. Training segmentation models on large images often needs GPUs and careful batch sizing.

11. Common Use Cases

- Medical image segmentation
- Satellite land-cover mapping
- Road/lane segmentation
- Cell/nuclei segmentation
- Defect localization

12. Common Mistakes

- Applying different augmentations to image and mask
- Using bilinear interpolation for class masks
- Shape mismatch during concatenation
- Ignoring class imbalance
- Evaluating only pixel accuracy

13. Edge Cases / Limitations

Small objects and fuzzy boundaries are difficult. U-Net may struggle with global context unless enhanced with attention or pretrained encoders.

14. Variations

Variation	What changes	When to use	Importance
ResUNet	Residual blocks	Deeper segmentation	High
Attention U-Net	Attention gates	Focus on target regions	Medium
U-Net+ +	Nested skip paths	Better fine detail	Medium
3D U-Net	Volumetric convs	CT/MRI volumes	High

15. Related Topics

U-Net vs autoencoder: U-Net keeps skip connections for spatial detail. U-Net vs FCN: U-Net has symmetric encoder-decoder with strong skips. ResNet can be used as U-Net encoder.

16. Interview Questions

1. What is U-Net used for?
Image segmentation and dense prediction.
2. Why skip connections?
They recover fine spatial details.
3. What is Dice coefficient?
Overlap metric between prediction and ground truth.
4. Why not use pixel accuracy alone?
It can be misleading with class imbalance.
5. What does 1x1 conv do at output?
Maps features to class logits per pixel.
6. What is transposed convolution?
Learnable upsampling.
7. Why is U-Net popular in medical imaging?
It works well with limited data and precise masks.
8. How handle multi-class segmentation?
Output C channels and use cross-entropy/Dice.
9. What causes mask augmentation bugs?
Different transforms or wrong interpolation.
10. U-Net vs ResNet?
U-Net is segmentation encoder-decoder; ResNet is usually classification backbone.

17. Practice Tasks

- Segment cells from microscopy images.
- Implement Dice loss.
- Debug mask interpolation artifacts.
- Train U-Net with a ResNet encoder.
- Compare BCE and Dice loss.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Tumor Segmentation	Segments tumors	PyTorch	BraTS	Healthcare AI
Road Extraction	Finds roads in satellite images	PyTorch, OpenCV	DeepGlobe	Geospatial CV
Defect Localizer	Produces defect masks	PyTorch	MVTec AD	Industrial inspection

19. Quick Revision

- Key idea: encoder-decoder segmentation with skip connections.
- Main formula: $\text{Dice} = 2 \cdot \text{intersection} / (\text{pred} + \text{target})$.
- When to use: pixel-level prediction.
- Important metrics: Dice, IoU.
- Common traps: mask augmentation and imbalance.
- Interview one-liner: U-Net combines context and localization for segmentation.

20. Final Cheat Sheet

Item	Answer
Definition	CNN encoder-decoder for segmentation
Input/output	Image to pixel mask
Main steps	Downsample, upsample, concatenate skips
Key hyperparameters	Depth, channels, loss, image size
Metrics	Dice, IoU, pixel F1
Pros	Excellent localization
Cons	High memory, limited global context
Best use cases	Medical and satellite segmentation